Pharmacy Inventory & Prescription Management System

Database Systems Lab (BCSE302P)

PROJECT REPORT

Fall Semester 2026-2027

Submitted by

ABHIRUP BANIK (24BCE0311)
SARVASWA BHANTI (24BCE0333)
AMOGH SHARMA (24BCE2126)
SAMI FARHAN SYED (24BCE2744)

in partial fulfillment for the award of the degree of

B. Tech

in

Computer Science and Engineering
Core



Vellore-632014, Tamil Nadu, India

School of Computer Science and Engineering

October, 2026

CONTENTS

Pharmacy Inventory & Prescription Management System

1. **Abstract** 3
2. **Introduction** 4
3. **Background Study** 6
4. **Methodology** 8
    - **4.1 Explanation of the Working Model and Dataset** 8
    - **4.2 Proposed Working Model and ER Diagram** 12
    - **4.3 Modules and Sub-modules** 19
5. **Results and Discussion** 23
6. **Conclusion** 55
7. **References** 56

1. ABSTRACT

A pharmacy has to get two things right every time a medicine is handed over. It must take the correct physical stock from the correct batch without ever losing count, and it must make sure that the medicine does not interact dangerously with the other medicines on the same prescription. These two needs pull in different directions. Stock and prescriptions need strict consistency, row-level locking and an audit trail that nobody can change afterwards. Drug interaction knowledge, on the other hand, has no fixed shape, comes from outside sources, and is read far more often than it is written.

In this project we built a working pharmacy inventory and prescription management system around that observation, using polyglot persistence, that is, a different database for each kind of data. PostgreSQL 17 is the transactional core. It holds medicines, batches, prescriptions and an append-only dispensing log, and it runs every dispense as one short ACID transaction. The transaction locks the prescription line and the batch, picks stock in First-Expiry-First-Out (FEFO) order through a partial B-tree index, and writes the stock change and the audit row together. MongoDB 9 is the safety layer. Its drug_interactions collection stores one document per drug pair, each with its own set of fields, protected by a $jsonSchema validator and a unique compound index. Every dispense checks MongoDB twice, once before and once inside the transaction, and if MongoDB cannot be reached the dispense is refused and rolled back.

The system runs as a Node.js and Express API with a React front end. On a seeded database of 60,181 batches the FEFO query takes under 0.2 ms with its index and 7 to 13 ms without it, two pharmacists competing for the last unit of a medicine are serialised correctly by the row lock, the audit log rejects every attempt to edit it, and all 24 automated end-to-end checks pass.

2. INTRODUCTION

2.1 Problem Background

Dispensing is the point at which a pharmacy's records meet the real world. The pharmacist reads a prescription, takes tablets from a shelf and hands them over. Behind this simple act there are several rules that the database has to enforce without exception:

- stock must never go below zero and must never be counted twice, even when two pharmacists work at the same time;
- the oldest stock that is still in date should be issued first, so that medicines do not expire on the shelf, and stock that has already expired must never be issued;
- a prescription must not be dispensed beyond the quantity the doctor wrote on it;
- every movement of stock must leave a record of what moved, from which batch, by whom and when, and that record must not be editable later.

Alongside these rules there is a second, quite different concern. Many medicines are dangerous in combination. Aspirin taken with warfarin raises the risk of bleeding, sildenafil taken with a nitrate can cause a severe fall in blood pressure, and clarithromycin taken with simvastatin can lead to muscle damage. Whether a combination matters, and what should be done about it, is clinical knowledge that differs from one pair to the next. Some interactions only matter above a certain dose, some only for elderly patients, some can be avoided by spacing the doses a few hours apart, and some combinations are prescribed on purpose.

2.2 Motivation for Two Databases

If the interaction knowledge is forced into a relational table, it either becomes a very wide table in which most columns are empty for most rows, or it is split into an attribute table that needs several joins for every lookup. Neither suits data that is read on every single dispense. The stock data has the opposite needs: its structure is fixed, but it must be protected by transactions, locks and constraints.

We therefore treat the two concerns as two kinds of data and give each one the database that suits it. This approach is known as polyglot persistence [1], [2]. PostgreSQL handles everything that must be exact and transactional, and MongoDB handles the knowledge whose shape varies. The main engineering problem is the point where the two meet: a dispense must never complete without the safety check, yet the transaction must stay short enough that pharmacists working in parallel do not block one another.

2.3 Objectives

1) To model medicines, batches, prescriptions and users relationally, with keys and CHECK constraints that make invalid data impossible to store.

2) To dispense in one ACID transaction that locks the prescription line and the batch, chooses the batch in FEFO order, and updates the stock and writes the audit row together, or rolls everything back.

3) To keep an audit log that the application itself is not able to rewrite.

4) To store drug interactions as flexible documents in MongoDB, check every pair of medicines on a prescription before dispensing, and refuse to dispense when the check cannot be made.

5) To make this behaviour visible and measurable: the transaction steps, two pharmacists competing for the same stock, a MongoDB outage, the query plan of the FEFO query and the audit-log guard can all be run from the application.

2.4 Scope

The system covers prescription entry, dispensing, receiving new stock, the inventory and expiry views, the audit log with a weekly report, and the interaction knowledge base. Billing, ordering from suppliers and user login are outside the scope of this project. The pharmacist performing an action is chosen from a list at the top of the screen, so that every audit row still records a real person.

2.5 Organisation of the Report

Section 3 reviews the work this project builds on. Section 4 explains the working model, the dataset, the database design with its ER diagram, and each module of the system. Section 5 presents the tests, the measured results, screenshots of every module and the main source code. Section 6 concludes the report and Section 7 lists the references.

3. BACKGROUND STUDY

3.1 The Relational Model and Transactions

Codd's relational model [3] is the basis for representing stock and prescriptions as normalised relations linked by keys. Silberschatz, Korth and Sudarshan [7] describe how normalisation removes redundancy and how integrity constraints keep stored data valid. The idea of a transaction as a unit of work that is atomic, consistent, isolated and durable (ACID) is set out in detail by Gray and Reuter [4]. In this project a single dispense, which reduces stock and writes an audit row, is exactly such a unit: either both changes happen or neither does.

3.2 Concurrency Control and Locking

When two transactions read and change the same row at the same time, one change can silently overwrite the other. Berenson et al. [5] showed that the standard SQL isolation levels do not all prevent such lost updates. A common and reliable remedy is to lock the rows that are about to be changed at the moment they are read, using SELECT ... FOR UPDATE. PostgreSQL documents this behaviour in its chapter on concurrency control [6]: a second transaction that wants the same row waits until the first one commits or rolls back, and then re-checks its condition against the new version of the row. Our dispense transaction relies on exactly this behaviour.

3.3 Indexing

B-tree indexes allow a database to find rows in sorted order without reading the whole table [7]. PostgreSQL also supports partial indexes [8], which only contain the rows that satisfy a condition. A pharmacy keeps the record of every batch it has ever received, but at any time only a small number of batches still hold stock. A partial index on batches that still have stock therefore stays small however long the system is in use.

3.4 Polyglot Persistence

Stonebraker and Cetintemel [9] argued that a single database design cannot serve every kind of workload well. Sadalage and Fowler [1] gave the name polyglot persistence to the practice of using several data stores, each for the data it suits best, and Kleppmann [2] discusses the main difficulty of this approach, which is keeping several systems consistent with each other. In this project we limit that difficulty by writing to only one system during a dispense: MongoDB is only read inside the dispensing path, never written, so no distributed commit is needed.

3.5 Document Databases and Schema Validation

Hecht and Jablonski [10] compare the main families of NoSQL stores and note that document stores suit records whose attributes vary from one record to the next. MongoDB adds optional schema validation using $jsonSchema [11]. A collection can require a minimum set of fields and

still accept any others, which gives flexibility without losing all control over the data. We use this for the interaction documents.

3.6 Drug Interaction Data and Stock Rotation

Public drug knowledge bases such as DrugBank [12] record drug-drug interactions with a description, a severity and advice on how to manage them, and the fields present depend on what is known about each pair. The 30 interaction documents in our seed data follow the same pattern. For stock handling, the World Health Organization's guidance on good storage and distribution practices [13] recommends that stock with the earliest expiry is issued first (FEFO) and that expired stock is kept apart and never issued. Both rules are enforced by the FEFO query in this project.

3.7 Summary

Table 1: Related work and how it is used in this project

Work	Contribution	Use in this project
Codd [3]; Silberschatz et al. [7]	Relational model, normalisation, B-tree indexes	Six-table schema in 3NF; FEFO index
Gray and Reuter [4]; Berenson et al. [5]	ACID transactions; isolation anomalies	One dispense is one transaction with row locks
PostgreSQL documentation [6], [8]	SELECT ... FOR UPDATE; partial indexes	Fixed lock order; idx_batches_fefo
Stonebraker and Cetintemel [9]; Sadalage and Fowler [1]; Kleppmann [2]	One database cannot fit every workload; polyglot persistence	PostgreSQL for stock, MongoDB for interactions
Hecht and Jablonski [10]; MongoDB documentation [11]	Document stores; schema validation	Variable-field documents with a validator
DrugBank [12]; WHO [13]	Interaction records; FEFO and expiry rules	Seed knowledge; FEFO and expired-stock handling

Most student and textbook pharmacy systems keep everything in one relational database and either leave out interaction checking or store it as a fixed table. What this project adds is a working combination of the two approaches, in which the safety check takes part in the stock transaction without the two databases needing a distributed commit.

4. METHODOLOGY

4.1 Explanation of the Working Model and Dataset

4.1.1 System Architecture

The system is built in three layers. The pharmacist only uses a React application in the browser. The browser sends HTTP requests with JSON bodies to an Express API, and only the API connects to the databases. Because of this, every rule is checked on the server or inside the database, and nothing depends on the browser behaving correctly. The API keeps two PostgreSQL connection pools, one for each database role described in Section 4.2.7, and one MongoDB client whose server-selection timeout is 1.5 seconds. The same Express server also serves the built React application, so the whole system opens at a single address, http://localhost:4000, and can be used from a laptop or from a phone on the same network.

Table 2: Technology stack

Layer	Technology	Role in the system
Transactional database	PostgreSQL 17.11	Stock, prescriptions and the audit log; transactions, row locks, constraints, indexes, views, trigger, roles
Safety database	MongoDB 9.0.2	drug_interactions documents; $jsonSchema validator; unique compound index
API server	Node.js 22, Express 4	20 REST routes; the two-phase dispense; serves the built front end
Database drivers	node-postgres 8, MongoDB driver 6	Explicit BEGIN, COMMIT and ROLLBACK; bounded connection timeout
Front end	React 18, Vite 6	Six screens; usable on laptop and mobile browsers
Tooling	setup.js, smoke.js, export.js	Rebuild both databases; 24 end-to-end checks; export data to CSV and JSON

4.1.2 Why Each Kind of Data Lives Where It Does

- **Medicines, batches, prescriptions and users** are fixed-shape records linked by keys, and changes to them must be atomic. They are stored in PostgreSQL.
- **The dispensing log** is stored in PostgreSQL as well, in the same database as the stock. This is what allows the stock change and its audit row to be committed in one transaction.
- **Drug interactions** have a different set of fields for each pair, are read on every dispense and are written rarely. They are stored in MongoDB.
- **Reports** such as current stock or the most dispensed medicines this week are not stored at all. They are PostgreSQL views, computed when they are read, so they can never be out of date.

4.1.3 The Dispensing Workflow

A dispense is split into two phases, a preview and a confirm. Splitting it this way is the most important design decision in the project, because it keeps locks away from the time a person spends reading the screen.

Preview. When the pharmacist picks a line of a prescription, the API reads the batches of that medicine in expiry order and works out which batches the requested quantity would be taken from. Expired batches are listed but never planned. At the same time the API takes the generic name of every medicine on the prescription, forms every pair, and asks MongoDB for all of those pairs in a single $or query. The result is shown on screen. Nothing is locked and nothing is written, so the pharmacist can read a warning for as long as needed, or cancel.

Confirm. When the pharmacist presses Confirm, the API opens one PostgreSQL transaction and performs these steps in this order:

1) BEGIN, then lock the prescription line with SELECT ... FOR UPDATE on prescription_items.

2) Lock the earliest unexpired batch that has stock, again with SELECT ... FOR UPDATE. If it holds less than the request, run the same query again, excluding the batches already taken, to lock the next batch. If the unexpired stock runs out, stop with an "insufficient stock" error.

3) Compare the request with what is still left on the prescription (prescribed minus already dispensed). A larger request is refused.

4) Check MongoDB again for every pair on the prescription. If MongoDB does not answer within 1.5 seconds the dispense is refused with HTTP 503. A contraindicated pair is refused outright, and a high or moderate warning that the pharmacist has not acknowledged is refused as well.

5) For each batch used, reduce its quantity with UPDATE and insert one DISPENSE row into dispensing_events.

6) COMMIT. The locks are released and the stock change and audit rows become visible to everyone at the same time.

If any step fails, the API issues ROLLBACK, which undoes every change made so far and releases every lock. Because every transaction locks the prescription line first and then batches in expiry order, two transactions can never be waiting for each other in a circle, so a deadlock cannot occur. Receiving new stock follows the same pattern: the new batch and a RECEIPT row in the audit log are inserted in one transaction.

4.1.4 Data Flow

Figure 1 shows, for every action the pharmacist can take, which API route is called, which PostgreSQL tables and views are read, written or locked, and what is asked of MongoDB. Each

row reads from left to right. The coloured tags show the kind of access: READ, WRITE, LOCK (a row lock held until COMMIT) and BLOCK (where the request can be refused and rolled back).

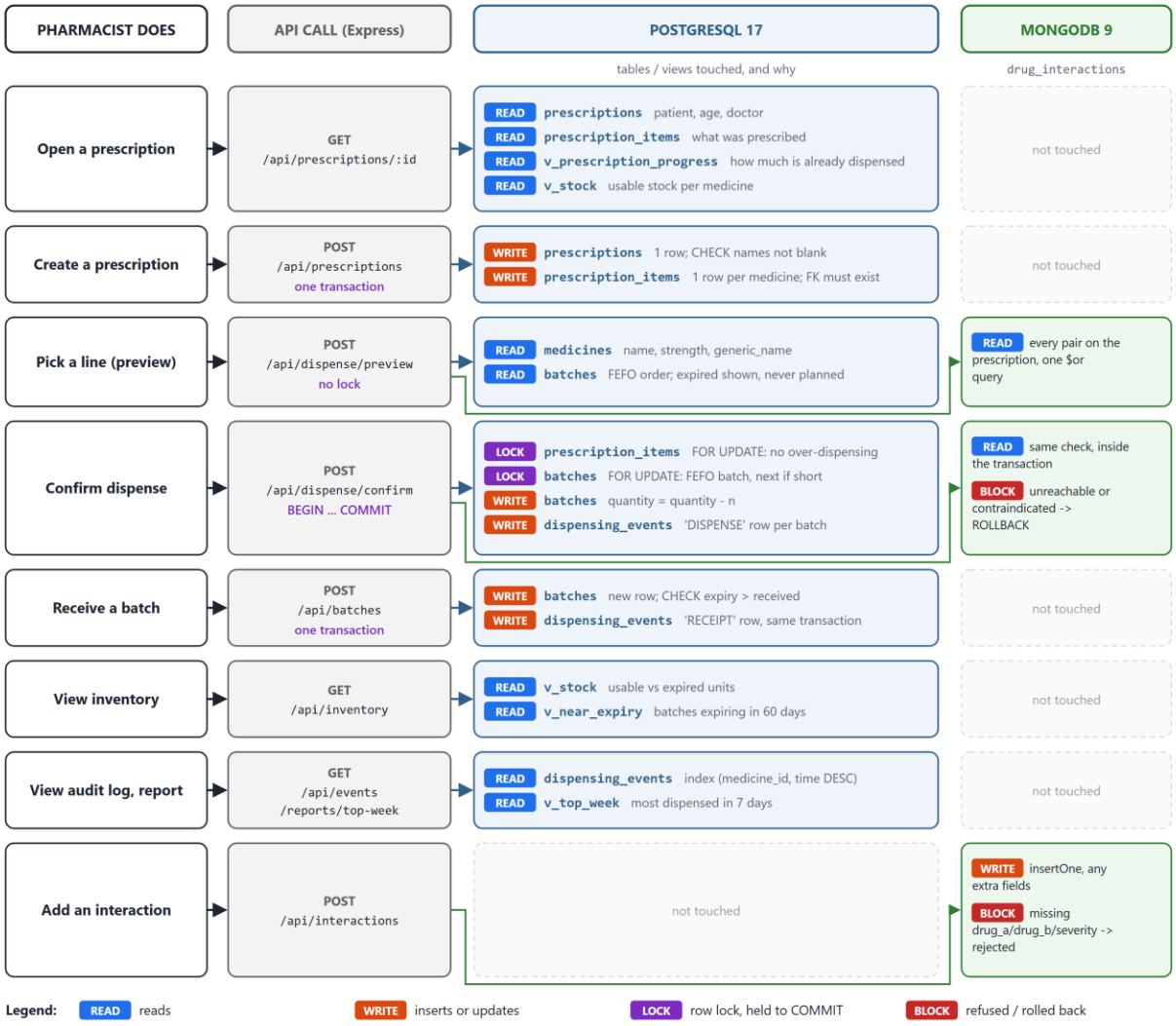


Figure 1: Data flow: for each action, the API call, the PostgreSQL tables touched and the MongoDB use

4.1.5 Dataset

The databases are created from scratch by a setup script (setup.js) in about two seconds, using the SQL files and a seed. All dates in the seed are calculated relative to the current date, so the data never goes out of date: there is always stock expiring soon, stock expiring later, and a few batches that have already expired.

The PostgreSQL side holds 44 real medicines with their generic names, and 181 batches that still hold stock, including one expired batch each for four of the medicines. It also holds 60,000

emptied historical batches. These have no stock, but they make the batches table large enough that the effect of the FEFO index can be measured. The ten seeded prescriptions are chosen to cover every case: one with no interactions, ones with high, moderate and low interactions, and one with a contraindicated pair. The audit log starts with 3,000 historical dispenses followed by 181 receipts, calculated so that the audit log adds up exactly to the stock on hand.

Table 3: Seed dataset

Data	Rows or documents	Purpose
medicines	44	Products with generic name, form, strength, pack and manufacturer
batches	181 with stock, 60,181 in total	Staggered expiry dates, expired lots, and history that makes the FEFO index matter
prescriptions / prescription_items	10 / 24	Every interaction severity, including a contraindicated pair
users	4	The four team members, three as pharmacists and one as admin
dispensing_events	3,181	History of dispenses and receipts that reconciles exactly with stock
drug_interactions (MongoDB)	30 documents using 18 different field names	Real interaction pairs whose fields vary

4.2 Proposed Working Model and ER Diagram

4.2.1 Proposed Working Model

Figure 2 shows the complete working model. The presentation layer has six screens. The API layer has three main files: index.js holds the routes, dispense.js holds the preview and the confirm transaction, and db.js holds the database connections. The data layer has six tables, four views, six indexes, an audit trigger and two roles in PostgreSQL, and one validated collection in MongoDB.

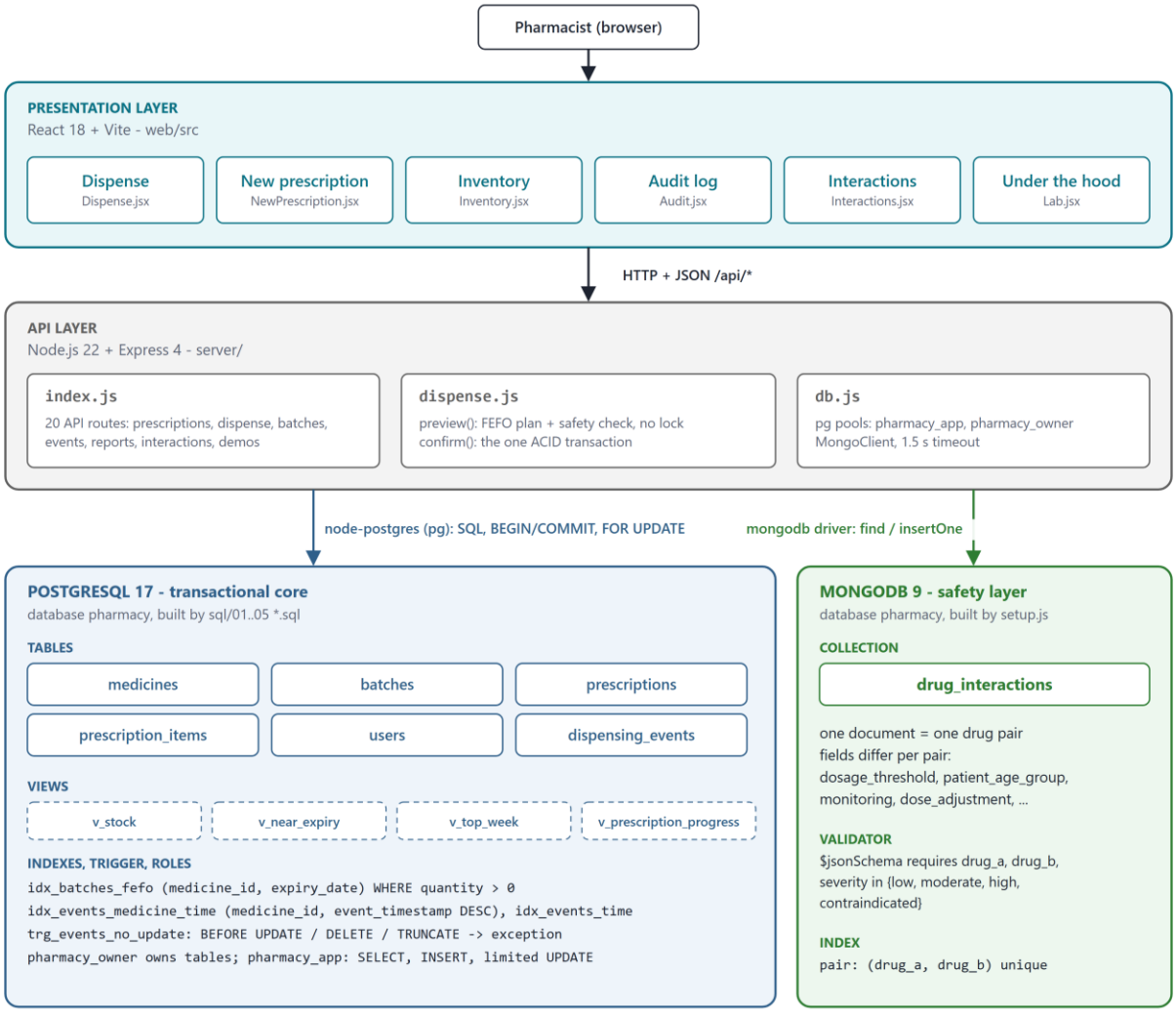


Figure 2: Proposed working model: three layers, the technology in each, and the objects in each database

4.2.2 ER Diagram

Figure 3 is the ER diagram of the system. It has five entities. A prescription contains many medicines, and a medicine appears on many prescriptions; this many-to-many relationship

carries its own attribute, the quantity prescribed, and becomes the prescription_items table. A medicine has many batches. Every dispensing event is performed by one user, takes stock from one batch, is of one medicine, and may cite one prescription (a walk-in sale or a stock receipt cites none).

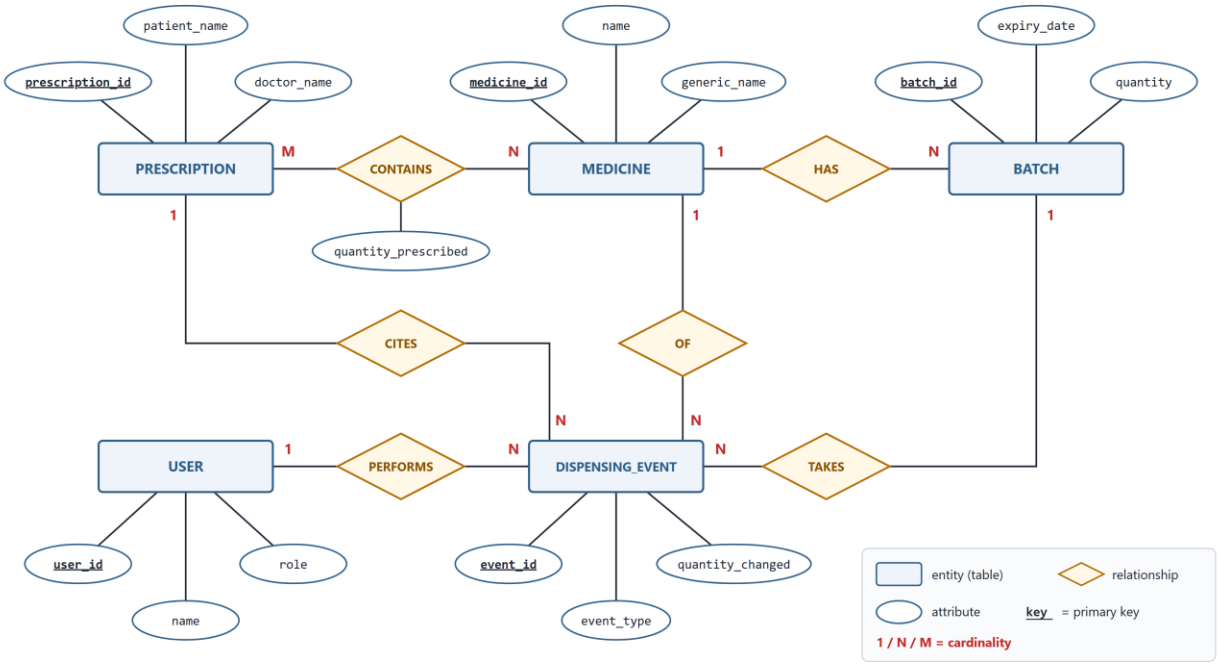


Figure 3: ER diagram of the system

Table 4: Relationships and how they are implemented

Relationship	Cardinality	Implementation
PRESCRIPTION contains MEDICINE	M : N	Table prescription_items with two foreign keys, UNIQUE (prescription_id, medicine_id) and attribute quantity_prescribed
MEDICINE has BATCH	1 : N	batches.medicine_id references medicines
USER performs DISPENSING_EVENT	1 : N	dispensing_events.performed_by references users, NOT NULL
DISPENSING_EVENT takes from BATCH	N : 1	dispensing_events.batch_id references batches, NOT NULL
DISPENSING_EVENT is of MEDICINE	N : 1	dispensing_events.medicine_id references medicines, NOT NULL
DISPENSING_EVENT cites PRESCRIPTION	N : 0..1	dispensing_events.prescription_id references prescriptions, nullable

4.2.3 Relational Schema

Figure 4 maps the ER diagram to the tables that are actually created, with every column, its type and its constraints. It also shows the trigger on the audit log, the four views, the indexes, and the MongoDB collection. The dashed green line marks the only value shared by the two databases: medicines.generic_name in PostgreSQL is the drug_a / drug_b key in MongoDB.

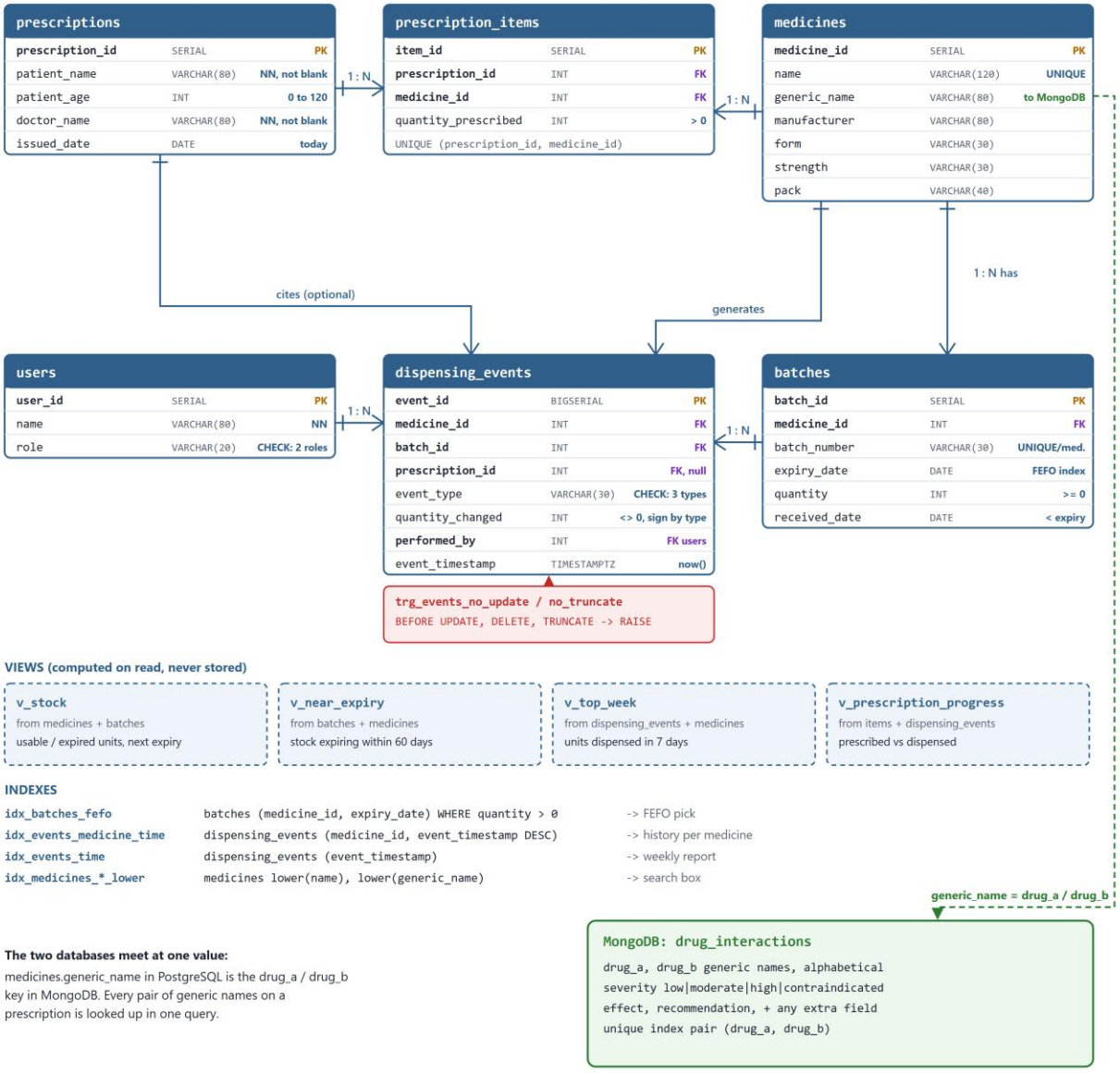


Figure 4: Relational schema with keys, constraints, trigger, views, indexes and the MongoDB collection

Table 5: What each table stores

Table	What it stores	Written by / read by
medicines	One row per product. generic_name links the medicine to MongoDB	Seeded once; read by search, preview and every report
batches	One row per physical lot: batch number, expiry date, quantity on hand, date received. This is the source of truth for stock	Inserted when stock is received; quantity reduced on dispense under a row lock
prescriptions	Patient name, age, doctor and date issued	Inserted from the New prescription form; read when a prescription is opened
prescription_items	One row per medicine on a prescription, with the quantity prescribed	Inserted with the prescription; locked on confirm
users	The pharmacists and the admin	Seeded; referenced by every audit row
dispensing_events	Append-only history of every DISPENSE, RECEIPT and ADJUSTMENT	Inserted with each stock change; never updated or deleted
drug_interactions (MongoDB)	One document per drug pair, with fields that vary	Read on every preview and every confirm; written rarely

Table 6: Constraints and the rule each one enforces

Constraint	Table	Rule enforced
CHECK (quantity >= 0)	batches	Stock can never go negative
CHECK (expiry_date > received_date)	batches	A batch cannot be received already expired
UNIQUE (medicine_id, batch_number)	batches	The same batch number cannot be entered twice for a medicine
CHECK (length(trim(name)) > 0)	prescriptions	Patient and doctor names cannot be blank
CHECK (patient_age BETWEEN 0 AND 120)	prescriptions	Age must be realistic
CHECK (quantity_prescribed > 0)	prescription_items	A line must prescribe something
UNIQUE (prescription_id, medicine_id)	prescription_items	A medicine appears once on a prescription
CHECK (event_type IN (...))	dispensing_events	Only DISPENSE, RECEIPT or ADJUSTMENT
CHECK on the sign of quantity_changed	dispensing_events	A dispense reduces stock and a receipt increases it

Constraint	Table	Rule enforced
CHECK (role IN (...))	users	Only pharmacist or admin
Foreign keys, ON DELETE RESTRICT	all	No row can refer to a medicine, batch, prescription or user that does not exist

4.2.4 Normalisation

The schema is in third normal form. Every column holds a single value, so it is in first normal form. Every table has a single-column surrogate primary key, so no column can depend on only part of a key, which gives second normal form. No non-key column depends on another non-key column: medicine details depend only on medicine_id, batch details only on batch_id, and patient details only on prescription_id. The quantity prescribed belongs to the pair of a prescription and a medicine, which is why it lives in prescription_items and not in either parent table. Total stock is deliberately not stored in medicines, because it would depend on the batches and could disagree with them; it is computed by the v_stock view instead.

4.2.5 Indexes and Views

Table 7: Indexes

Index	Definition	Query it serves
idx_batches_fefo	batches (medicine_id, expiry_date) WHERE quantity > 0	The FEFO query: the earliest batch of a medicine that has stock
idx_events_medicine_time	dispensing_events (medicine_id, event_timestamp DESC)	Recent history of one medicine, newest first
idx_events_time	dispensing_events (event_timestamp)	The weekly report
idx_items_prescription	prescription_items (prescription_id)	Opening a prescription
idx_medicines_name_lower, idx_medicines_generic_lower	medicines (lower(name)), (lower(generic_name))	Case-insensitive medicine search

In idx_batches_fefo, medicine_id comes first because the query looks for one medicine, and expiry_date comes second because the query wants the earliest date, so the first matching index entry is the answer. The WHERE clause makes it a partial index: only batches that still hold stock are stored in it, so emptied batches never slow it down.

Table 8: Views

View	Computed from	What it gives
v_stock	medicines, batches	Usable and expired units, next expiry date and number of live batches per medicine
v_near_expiry	batches, medicines	Batches with stock that expire within 60 days, with days left
v_top_week	dispensing_events, medicines	Units dispensed per medicine in the last seven days
v_prescription_progress	prescription_items, dispensing_events	Quantity prescribed against quantity already dispensed

4.2.6 MongoDB Document Design

Each document in drug_interactions describes one pair of drugs by their generic names. Only three fields are required: drug_a, drug_b and severity, which must be low, moderate, high or contraindicated. The $jsonSchema validator enforces these and accepts any other field. The pair is always stored in alphabetical order (drug_a comes before drug_b), and a unique compound index named pair covers (drug_a, drug_b). Together these mean that a pair can only be stored once and can be found with one index lookup whichever order the medicines appear in. The stored document for aspirin and warfarin is shown below; it carries two fields, mechanism and monitoring, that many other documents do not have.

```
{
  drug_a:         "Aspirin",
  drug_b:         "Warfarin",
  severity:       "high",
  effect:         "Increased risk of bleeding",
  recommendation: "Monitor patient closely. Consider dose
                   adjustment or alternative therapy.",
  mechanism:      "Additive antiplatelet and anticoagulant effect",
  monitoring:     ["INR", "signs of bleeding"]
}
```

Severity decides what the dispense does. A contraindicated pair blocks the dispense completely. A high or moderate pair must be acknowledged by ticking a box on the warning, and the acknowledgement is checked again on the server. A low pair is shown for information only.

4.2.7 Roles and Protection of the Audit Log

Two PostgreSQL roles are used. pharmacy_owner owns every table, view and function. pharmacy_app is the role the API connects as, and it is given only the privileges it needs:

Table 9: Privileges of the API role (pharmacy_app)

Object	Privileges	Reason
All tables and views	SELECT	The API reads everything
batches, prescriptions, prescription_items	INSERT, UPDATE	Receiving stock, dispensing, creating prescriptions
dispensing_events	INSERT only	The audit log can be added to but not changed
Sequences	USAGE	Generating primary keys

The audit log is protected twice. First, pharmacy_app has no UPDATE or DELETE privilege on dispensing_events, so any such statement from the API fails with a permission error. Second, the owner does have those privileges, so a trigger function, reject_audit_change(), runs BEFORE UPDATE, DELETE or TRUNCATE on the table and raises an error for everyone. As a result, nobody using the database normally can rewrite the history.

4.3 Modules and Sub-modules

The system is divided into six modules that share a common platform layer. Figure 5 shows each module with its sub-modules and the tables or collection it uses.

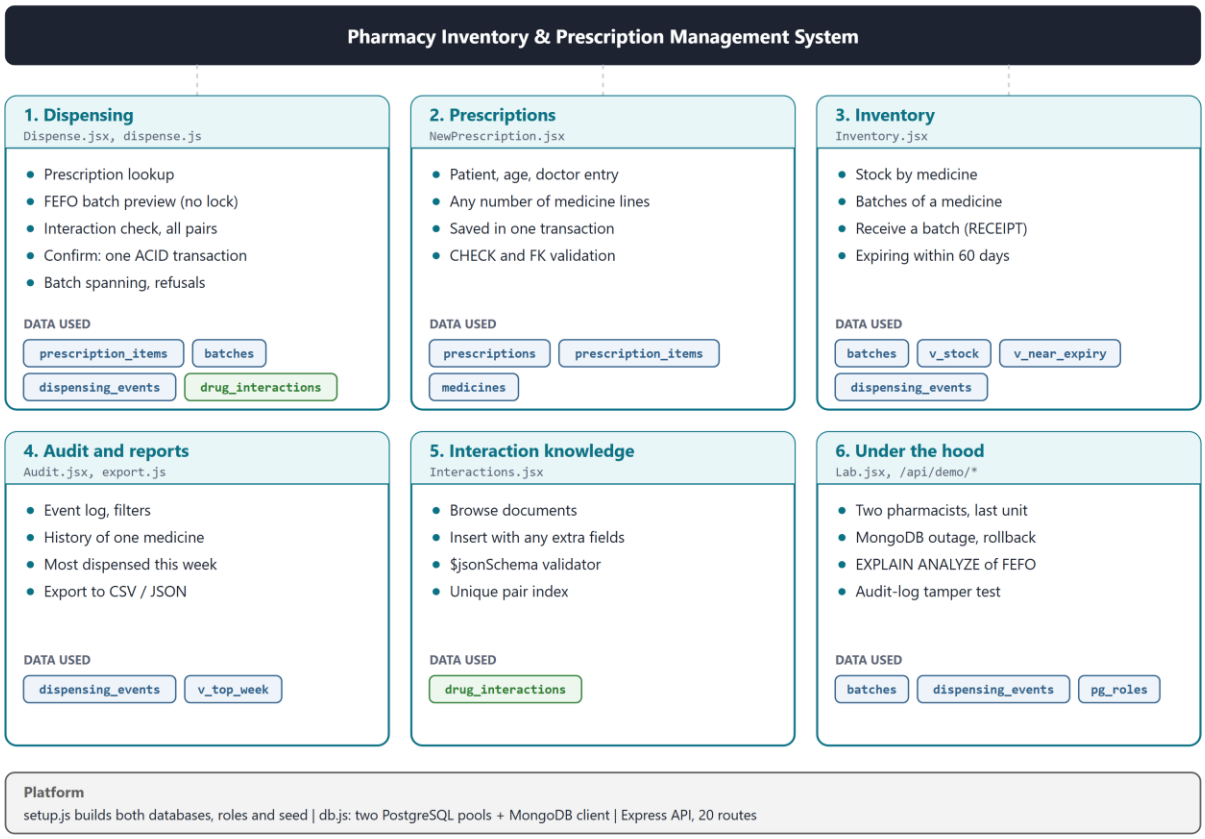


Figure 5: Modules, sub-modules and the data each one uses

4.3.1 Dispensing

This is the core module. When a prescription is opened, each line shows the quantity prescribed, the quantity already dispensed (from v_prescription_progress) and the stock available (from v_stock). Picking a line runs the preview described in Section 4.1.3: the batches of the medicine are listed in expiry order, the batch FEFO will take from is starred, and expired batches are struck through and marked as never dispensed. If the request is larger than the first batch, the screen shows how much each batch will give. The warning panel lists every interaction found on the prescription with its recommendation and any extra fields such as mechanism or monitoring. Confirm stays disabled until any high or moderate warning is acknowledged, and is not offered at all for a contraindicated pair. After a confirm, the screen shows every step of the transaction with its time in milliseconds. Figure 6 shows the transaction in detail, including the three ways it can end in ROLLBACK.

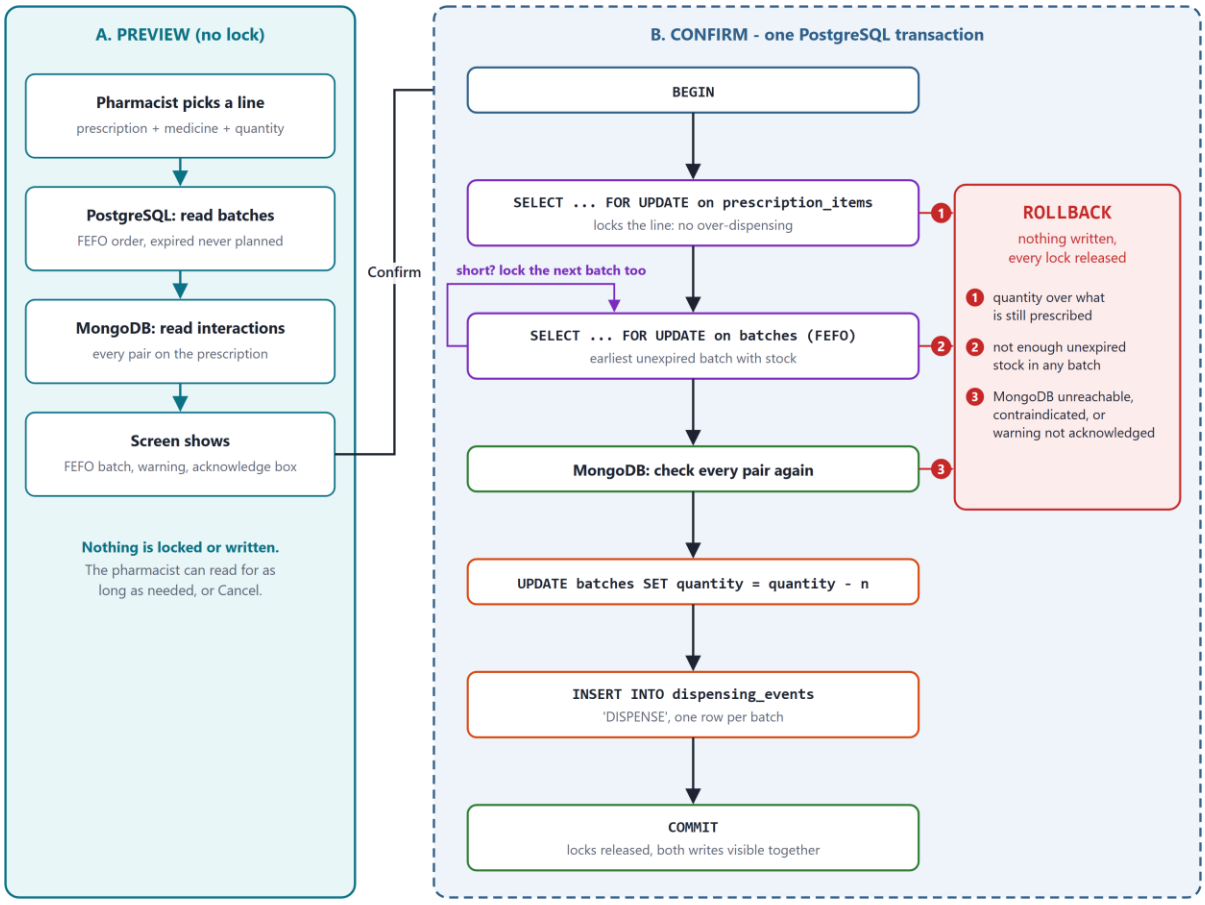


Figure 6: The dispense: a lock-free preview, then one short confirm transaction

4.3.2 Prescriptions

The New prescription form takes the patient name, age, doctor and any number of medicine lines. Saving it inserts one row into prescriptions and one row per line into prescription_items in

a single transaction. If anything is wrong, for example a blank patient name, an impossible age or a medicine that does not exist, a CHECK or foreign-key constraint rejects it and the whole transaction is rolled back, so a prescription is never saved half-way. A saved prescription appears at once in the Dispense screen and its interactions are checked as soon as a line is picked.

4.3.3 Inventory

The Inventory screen lists the stock of every medicine from v_stock, with usable and expired units in separate columns. Clicking a medicine shows all its batches. The Receive a batch form takes a batch number, expiry date and quantity, and inserts the batch together with a RECEIPT row in the audit log in one transaction. The CHECK on expiry date rejects a batch that would already be expired, and the UNIQUE constraint rejects a repeated batch number. A second list shows all stock expiring within 60 days, from v_near_expiry.

4.3.4 Audit and Reports

The Audit log screen shows dispensing_events newest first, with filters for medicine and event type. Filtering by medicine uses the index on (medicine_id, event_timestamp DESC), so the history of one medicine is found without scanning the table. Beside the log, a bar chart shows the medicines dispensed most in the last seven days, from v_top_week. All data can also be exported to CSV and JSON files with export.js.

4.3.5 Interaction Knowledge

The Interactions screen shows every document in drug_interactions, coloured by severity. A text box allows a new document to be inserted in JSON form. Any extra fields are accepted, so new kinds of clinical guidance can be added without changing any schema. A document that is missing drug_a, drug_b or a valid severity is rejected by the validator, and a pair that already exists is rejected by the unique index.

4.3.6 Under the Hood

This screen runs four demonstrations of the database concepts on the live database, each showing the SQL it runs:

- **Two pharmacists, the last unit.** The oldest Cetirizine batch is set to exactly one unit. Transaction A locks it and holds the lock for two seconds; transaction B asks for the same unit 250 ms later. The screen shows both timelines.
- **MongoDB unreachable.** The API's MongoDB driver is pointed at a closed port, exactly as if the server were down, and a dispense is attempted; then MongoDB is restored and the same dispense is tried again.
- **Why the FEFO query stays fast.** EXPLAIN ANALYZE runs the FEFO query twice, once normally and once with index scans switched off for that transaction, and shows both plans and times.

➔ **Can the audit log be rewritten.** The same UPDATE on the newest audit row is tried as pharmacy_app and as pharmacy_owner.

4.3.7 Platform and API

The platform layer is shared by all modules. setup.js creates the roles, the database, the tables, indexes, trigger and views, grants the privileges, creates the MongoDB collection with its validator and index, and loads the seed. db.js manages the connections. The API exposes the following routes:

Table 10: API routes

Route	Purpose
GET /api/health	Status of both databases
GET /api/users	Pharmacists and admin for the user selector
GET /api/medicines?q=	Medicine search by name or generic name
GET /api/prescriptions, GET /api/prescriptions/:id	List prescriptions; open one with its lines and progress
POST /api/prescriptions	Create a prescription and its lines in one transaction
POST /api/dispense/preview	FEFO plan and interaction check, no lock
POST /api/dispense/confirm	The dispense transaction; returns every step
GET /api/inventory, GET /api/inventory/near-expiry	v_stock and v_near_expiry
GET /api/medicines/:id/batches	All batches of one medicine
POST /api/batches	Receive a batch with its RECEIPT event
GET /api/events, GET /api/reports/top-week	Audit log with filters; weekly report
GET /api/interactions, POST /api/interactions	Read and insert MongoDB documents
POST /api/demo/race, GET /api/demo/explain, POST /api/demo/tamper, POST /api/demo/mongo	The four demonstrations

Every query sent to PostgreSQL is parameterised ($1, $2, ...), so user input is never joined into SQL text. Database errors are translated into clear HTTP responses: a unique violation becomes 409, and CHECK, foreign-key and not-null violations become 422 with the reason shown on screen.

5. RESULTS AND DISCUSSION

5.1 Testing

Every feature was tested in two ways. First, each module was used by hand through the screens, as shown in Section 5.4. Second, a test script, smoke.js, sends real requests to the running API and checks 24 conditions, from the FEFO choice to the audit-log guard. The script is run after every change. All 24 checks pass.

Table 11: Automated end-to-end checks (smoke.js)

No.	Check	Result
1	Health: both databases reachable	Pass
2	Preview finds the Aspirin and Warfarin warning on prescription 1	Pass
3	FEFO picks the earliest unexpired batch	Pass
4	Expired batch is listed but never planned	Pass
5	Confirm without acknowledging the warning: 409 and ROLLBACK	Pass
6	Confirm with acknowledgement: COMMIT	Pass
7	Stock goes down by exactly the quantity dispensed	Pass
8	Dispensing beyond the prescription is refused	Pass
9	Contraindicated Sildenafil and Isosorbide is refused	Pass
10	A request larger than one batch spans two batches with two audit rows	Pass
11	Insufficient stock: 409 and ROLLBACK	Pass
12	Race: both pharmacists succeed, B takes the next batch	Pass
13	Race: B gets the lock only after A commits	Pass
14	MongoDB down: 503 and ROLLBACK	Pass
15	MongoDB back: the same dispense commits	Pass
16	EXPLAIN uses idx_batches_fefo	Pass
17	Without the index the plan is a sequential scan	Pass
18	Audit UPDATE refused for pharmacy_app (privilege)	Pass
19	Audit UPDATE refused for pharmacy_owner (trigger)	Pass
20	MongoDB validator rejects a document without the required fields	Pass
21	Receiving a batch writes the batch and a RECEIPT row	Pass
22	CHECK rejects a batch that expires before it is received	Pass
23	A new prescription is saved with two lines	Pass
24	CHECK rejects a blank patient name and nothing is saved	Pass

5.2 Measured Results

Table 12 gives the measured behaviour of the four database concepts the project is built around. Timings were taken on a laptop with both databases running locally; the FEFO timings are the range over five runs.

Table 12: Measured results

Test	Result	What it shows
FEFO query, EXPLAIN ANALYZE	0.04 to 0.16 ms with the partial index; 7.2 to 12.9 ms with index scans off	The index makes the query 50 to 200 times faster
Buffers read by the FEFO query	3 with the index; 443 with a sequential scan, which filters out over 60,000 rows	Only batches with stock are in the index
Two pharmacists, last unit	B waits from about 255 ms to about 2,025 ms, gets the lock within 1 ms of A's COMMIT, then takes the next batch	The row lock serialises the two; the unit is sold once
MongoDB unreachable	HTTP 503 and ROLLBACK after the 1.5 s timeout; the same dispense commits once MongoDB is back	There is no dispense without the safety check
Audit log rewrite	pharmacy_app: error 42501, permission denied. pharmacy_owner: stopped by the trigger	The audit log is append-only by the database itself
Audit reconciliation	For every batch, the sum of quantity_changed equals the quantity on hand	History and stock agree

5.3 Discussion

Short transactions. The preview and confirm split was the most important decision. An earlier version of the design held the batch lock while the warning was on screen, which would have blocked every other pharmacist working on the same medicine for as long as one person was reading. With the split, a lock is held only for the few tens of milliseconds the transaction takes. The race test shows that correctness is not lost: the second transaction waits, re-reads the row after the first commits, finds it empty and moves on to the next batch.

Index design. The partial index is the reason the FEFO query stays fast as history grows. Emptied batches never enter the index, so its size depends on the stock on the shelf and not on how long the pharmacy has been running. Without the index PostgreSQL has to read and discard more than 60,000 rows to find one.

Two databases without a distributed commit. Keeping MongoDB out of the write path removed the need for any commit across the two databases. A dispense writes only to PostgreSQL, and MongoDB is a read that must succeed before the transaction is allowed to commit. The price of polyglot persistence is a second system that must be available; our design chooses safety and refuses to dispense when it is not, which the outage test confirms.

Safety in the database, not the screen. The screen disables Confirm for an unacknowledged or contraindicated warning, but the same rules are checked again on the server inside the transaction, and the constraints and the audit trigger hold even for someone with direct database access. The tests in Table 11 send requests that bypass the screen, and every unsafe request is refused.

Limitations. User login is not implemented, the interaction data is a curated set of 30 pairs rather than a licensed drug database, and the system runs on one machine. None of these change the database design, which is the subject of this project, and each is a natural next step.

5.4 Screenshots of All Modules

The screenshots below follow one complete session on the seeded data, module by module. Each one is preceded by a short note on what is being done.

5.4.1 Dispensing

The Dispense screen at start. The header shows that both PostgreSQL and MongoDB are reachable and which pharmacist is acting.

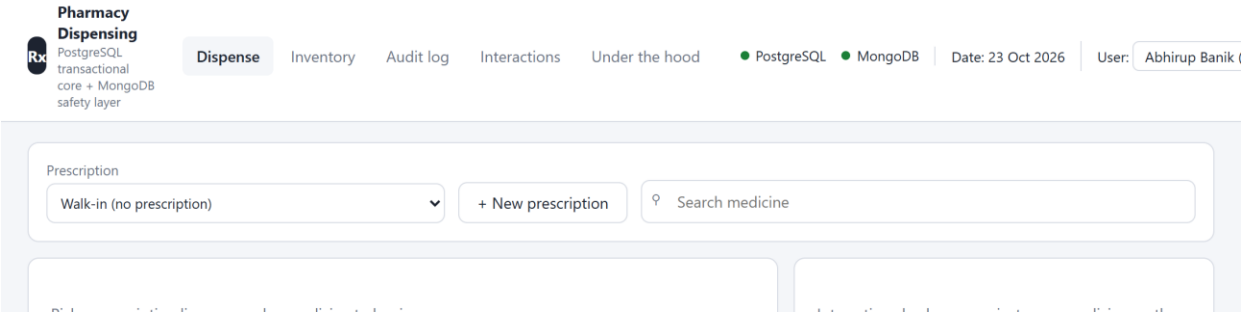


Figure 7: The Dispense screen at start; both databases reachable

Prescription 1 is opened and the Aspirin line is picked. The expired batch is struck through, the earliest unexpired batch is starred, and the Aspirin and Warfarin warning must be acknowledged.

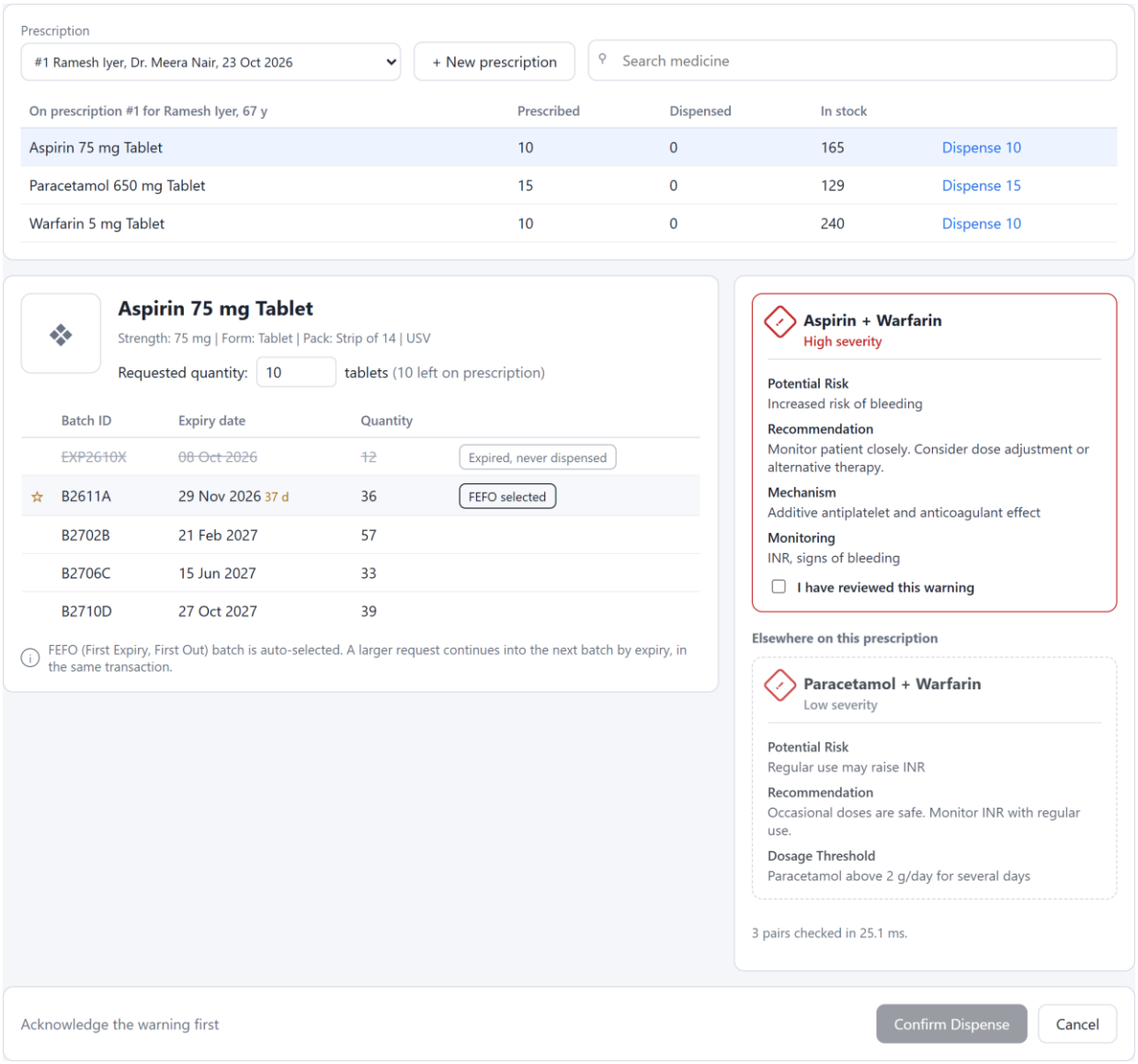


Figure 8: Preview: FEFO batch starred, expired batch struck through, warnings for every pair

After acknowledging the warning, Confirm runs the transaction. Each step is listed with its time, from BEGIN to COMMIT.

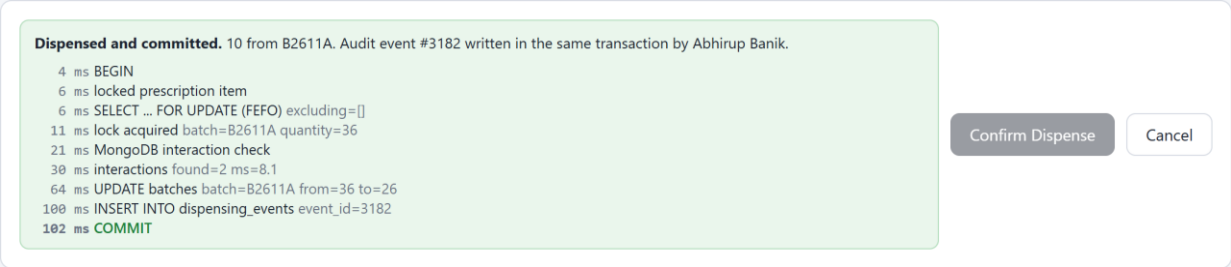


Figure 9: Confirm: the transaction steps from BEGIN to COMMIT

The prescription now shows Aspirin as dispensed and the stock reduced by ten.

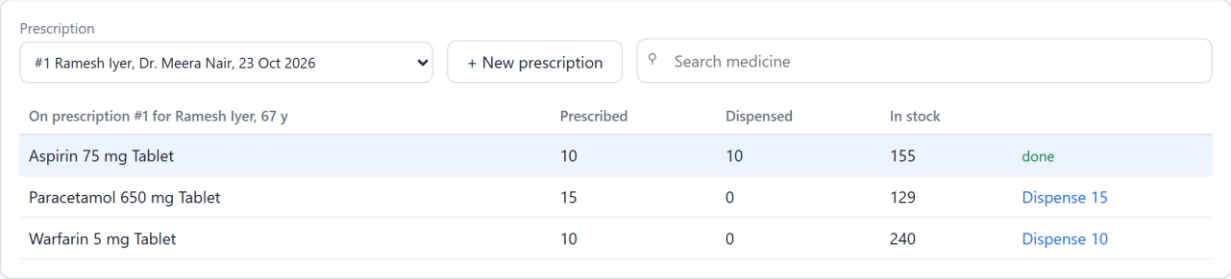


Figure 10: The prescription after the dispense

The Warfarin line raises the same warning from the other side. Until it is acknowledged, Confirm stays disabled.

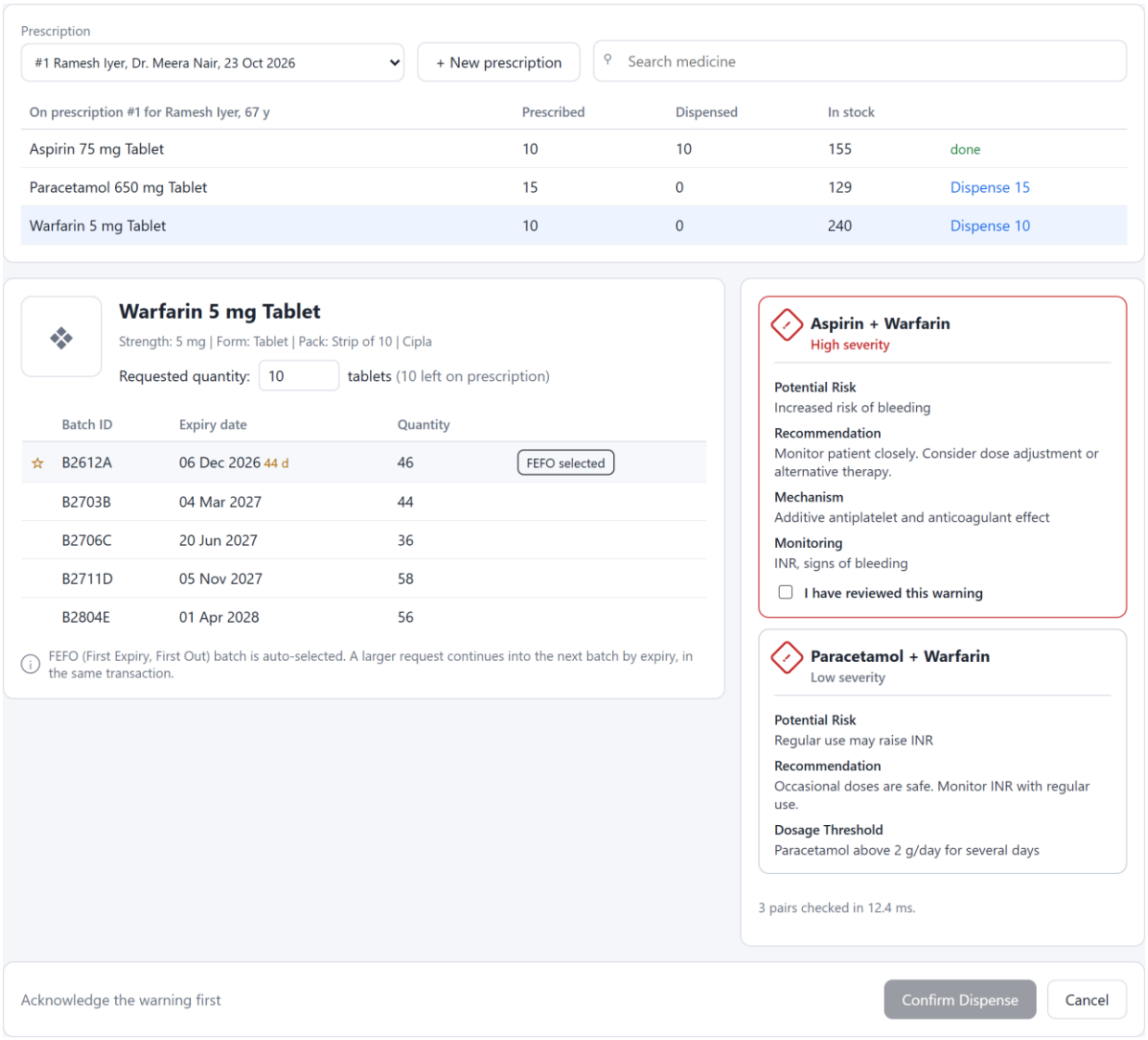


Figure 11: An unacknowledged high-severity warning blocks Confirm

Prescription 4 combines Sildenafil with Isosorbide Mononitrate. The pair is contraindicated, so there is nothing to acknowledge and the dispense is blocked.

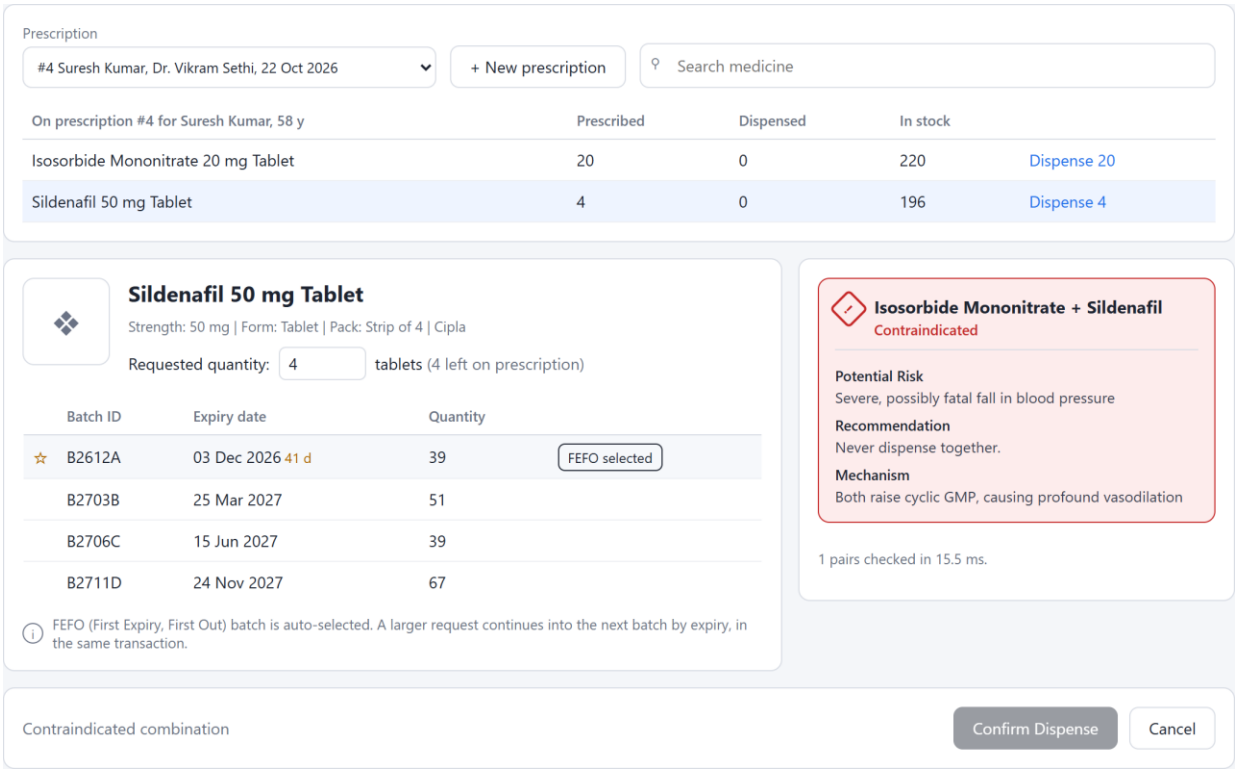


Figure 12: A contraindicated combination cannot be dispensed

Thirty-two tablets of Montelukast are requested but the FEFO batch holds fewer. The plan takes what the first batch has and the rest from the next batch.

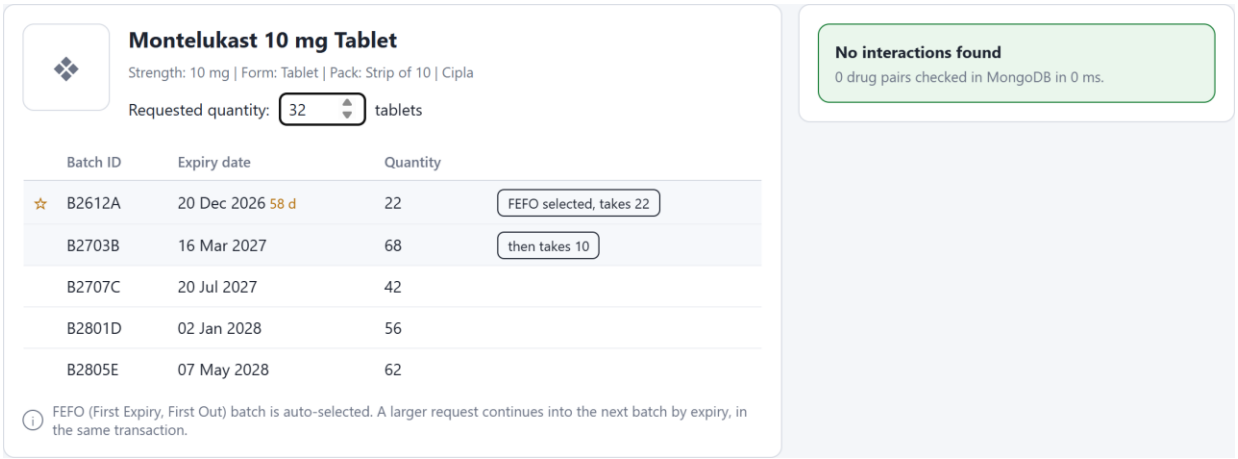


Figure 13: A request larger than one batch: the plan spans two batches

Confirm locks both batches in expiry order and writes one audit row for each.

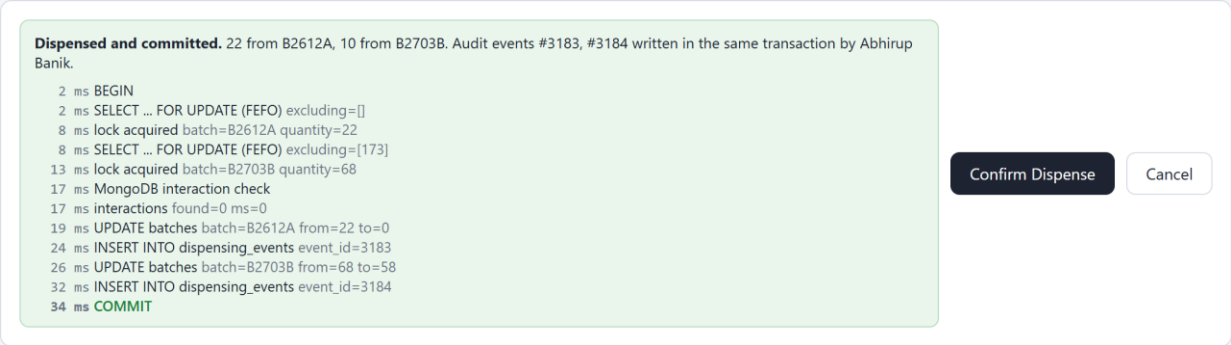


Figure 14: Committed across two batches, one audit row per batch

5.4.2 Prescriptions

A new prescription is entered for a new patient with two medicines, Warfarin and Fluconazole.

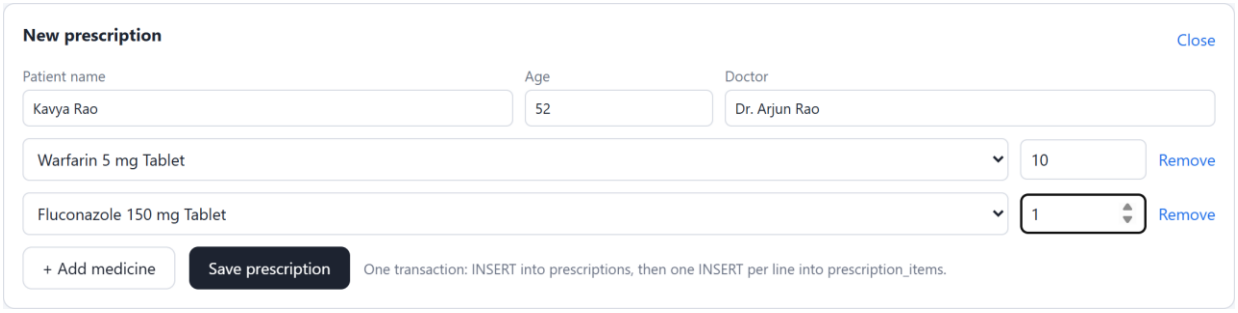


Figure 15: New prescription form with two medicine lines

Saving inserts the prescription and both lines in one transaction, and the prescription opens with the stock of each line.

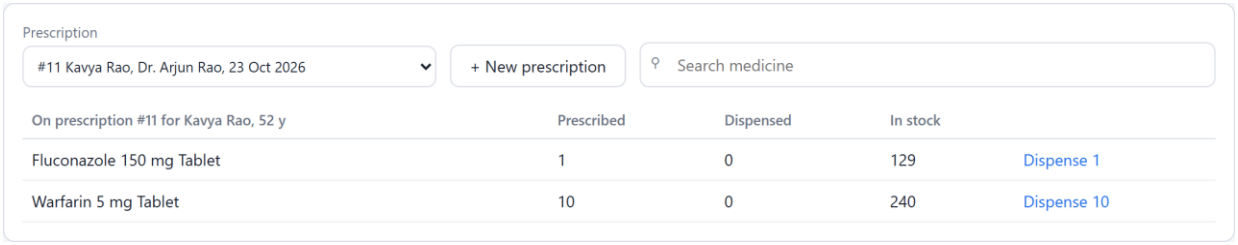


Figure 16: Prescription saved in one transaction and opened

Picking the Fluconazole line immediately finds the Fluconazole and Warfarin interaction on the new prescription.

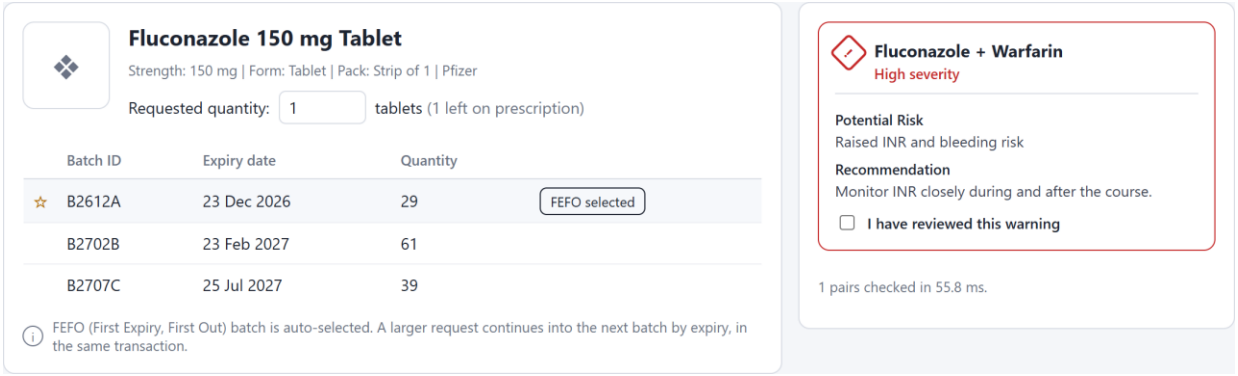


Figure 17: The new prescription raises its interaction warning at once

5.4.3 Inventory

A new Paracetamol batch is received with its batch number, expiry date and quantity.

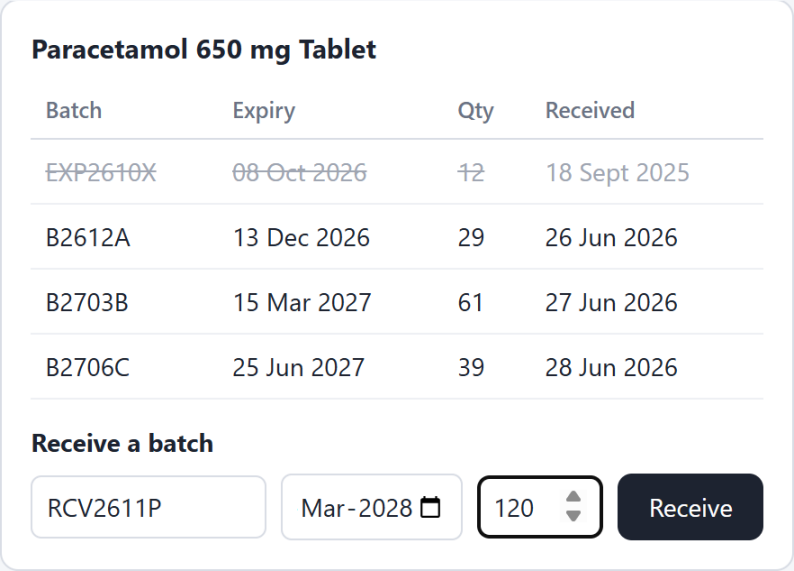


Figure 18: Receiving a new batch

The batch is inserted together with a RECEIPT row in the audit log, and appears in the batch list.

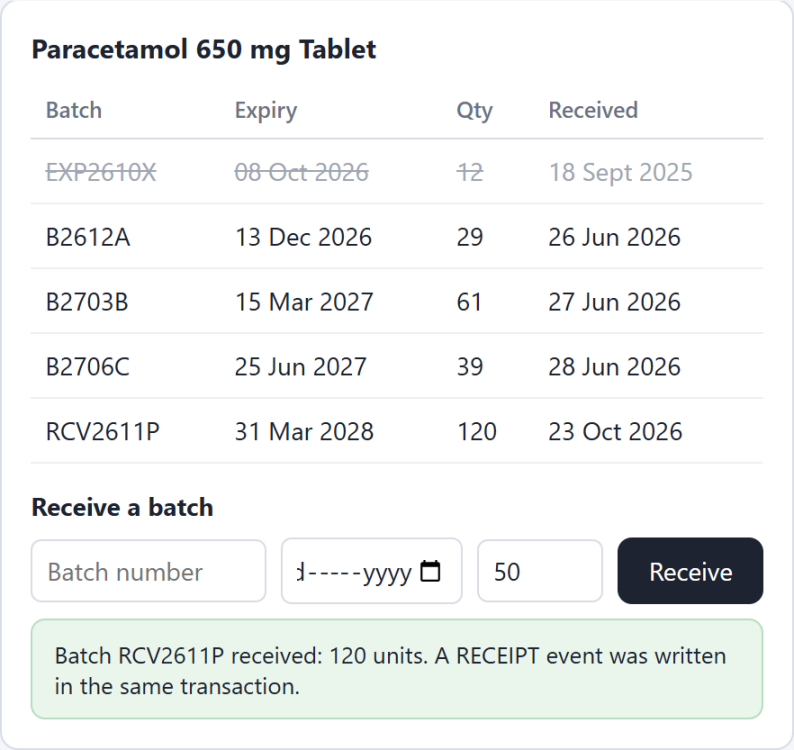


Figure 19: Batch received; RECEIPT written in the same transaction

Stock by medicine comes from the v_stock view. Expired units are counted in their own column and are never offered for dispensing.



Stock by medicine

Filter

From the view v_stock. Expired units are counted apart and never offered for dispensing.

Medicine	Usable	Expired	Next expiry	Batches
Allopurinol 100 mg Tablet	196	0	28 Dec 2026	4
Amiodarone 200 mg Tablet	180	0	07 Dec 2026	4
Amlodipine 5 mg Tablet	126	12	26 Nov 2026	4
Amoxicillin 500 mg Capsule	166	0	13 Dec 2026	4
Aspirin 75 mg Tablet	155	12	29 Nov 2026	5
Atorvastatin 10 mg Tablet	180	0	22 Dec 2026	4
Azathioprine 50 mg Tablet	225	0	25 Nov 2026	5
Azithromycin 500 mg Tablet	147	0	25 Dec 2026	3
Calcium Carbonate 500 mg Tablet	182	0	09 Dec 2026	4
Carbamazepine 200 mg Tablet	225	0	29 Nov 2026	5
Cetirizine 10 mg Tablet	126	0	06 Dec 2026	3
Ciprofloxacin 500 mg Tablet	135	0	17 Dec 2026	3
Clarithromycin 500 mg Tablet	215	0	29 Dec 2026	5
Clopidogrel 75 mg Tablet	138	0	24 Nov 2026	3
Digoxin 0.25 mg Tablet	270	0	14 Dec 2026	5
Fluconazole 150 mg Tablet	129	0	23 Dec 2026	3
Fluoxetine 20 mg Capsule	220	0	12 Dec 2026	5
Furosemide 40 mg Tablet	141	0	21 Dec 2026	3
Glimepiride 2 mg Tablet	198	0	30 Dec 2026	4
Hydroxychloroquine 200 mg Tablet	250	0	18 Dec 2026	5
Ibuprofen 400 mg Tablet	178	12	20 Dec 2026	5
Isosorbide Mononitrate 20 mg Tablet	220	0	10 Dec 2026	5

Figure 20: Stock by medicine from v_stock: usable and expired units

All stock expiring within 60 days, with the number of days left, comes from v_near_expiry.

Expiring within 60 days

Batch	Medicine	Expiry	Qty
~~EXP2610X~~	~~Ibuprofen 400 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Amlodipine 5 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Aspirin 75 mg Tablet~~	~~expired~~	12
~~EXP2610X~~	~~Paracetamol 650 mg Tablet~~	~~expired~~	12
B2611A	Rifampicin 450 mg Capsule	30 d	30
B2611A	Methotrexate 2.5 mg Tablet	31 d	49
B2611A	Clopidogrel 75 mg Tablet	32 d	38
B2611A	Azathioprine 50 mg Tablet	33 d	37
B2611A	Amlodipine 5 mg Tablet	34 d	26
B2611A	Metformin 500 mg Tablet	35 d	25
B2611A	Lisinopril 10 mg Tablet	36 d	44
B2611A	Carbamazepine 200 mg Tablet	37 d	43
B2611A	Aspirin 75 mg Tablet	37 d	26
B2611A	Trimethoprim 100 mg Tablet	38 d	32
B2612A	Omeprazole 20 mg Capsule	39 d	21
B2612A	Levothyroxine 50 mcg Tablet	40 d	20
B2612A	Sildenafil 50 mg Tablet	41 d	39
B2612A	Prednisolone 10 mg Tablet	42 d	38
B2612A	Losartan 50 mg Tablet	43 d	27
B2612A	Warfarin 5 mg Tablet	44 d	46
B2612A	Cetirizine 10 mg Tablet	44 d	26
B2612A	Amiodarone 200 mg Tablet	45 d	45
B2612A	Pantoprazole 40 mg Tablet	46 d	34

Figure 21: Stock expiring within 60 days from v_near_expiry

5.4.4 Audit and Reports

The audit log filtered to receipts shows the batch just received at the top, with who received it and when. The bars show the most dispensed medicines this week.

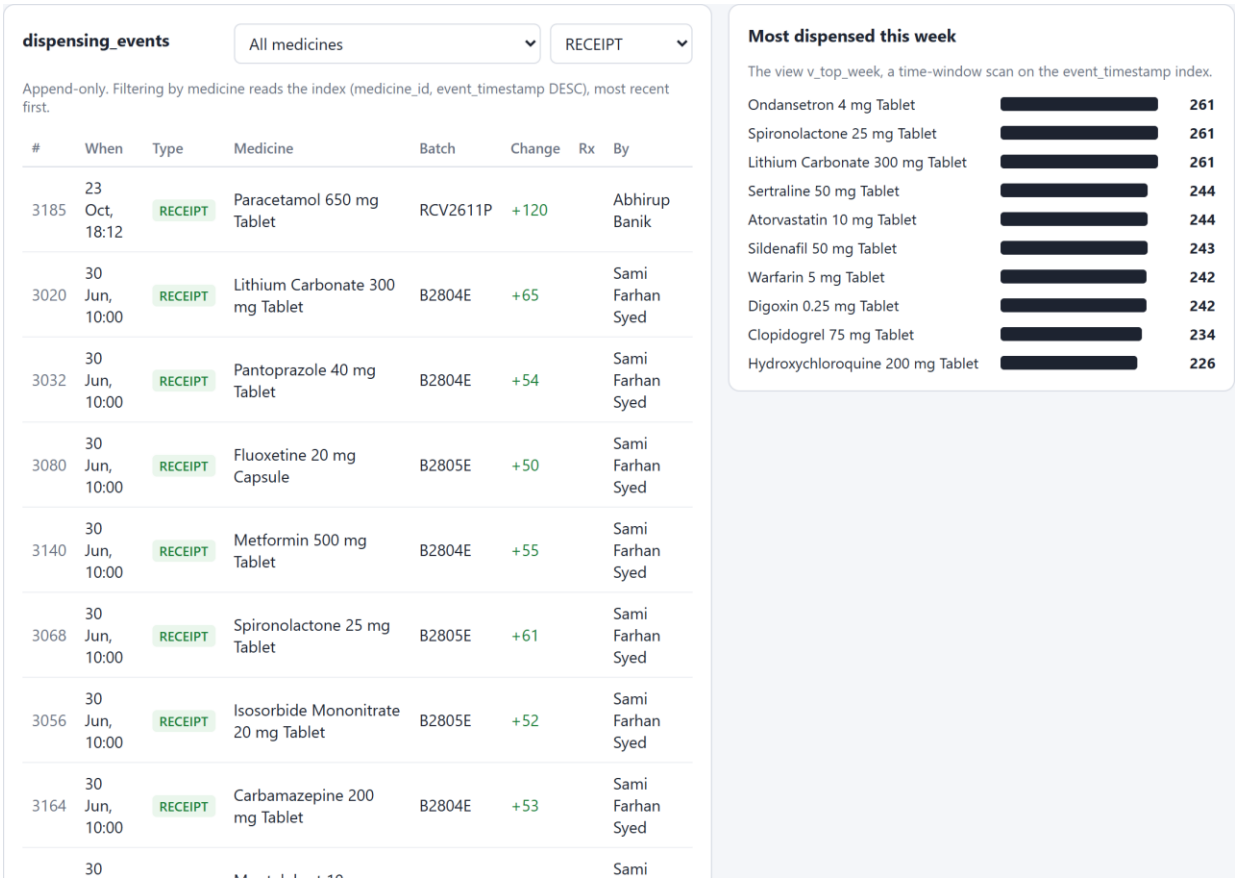


Figure 22: Audit log filtered to receipts, with the most dispensed medicines this week

Filtering to Aspirin reads the index on (medicine_id, event_timestamp DESC); the dispense made earlier is the newest row.

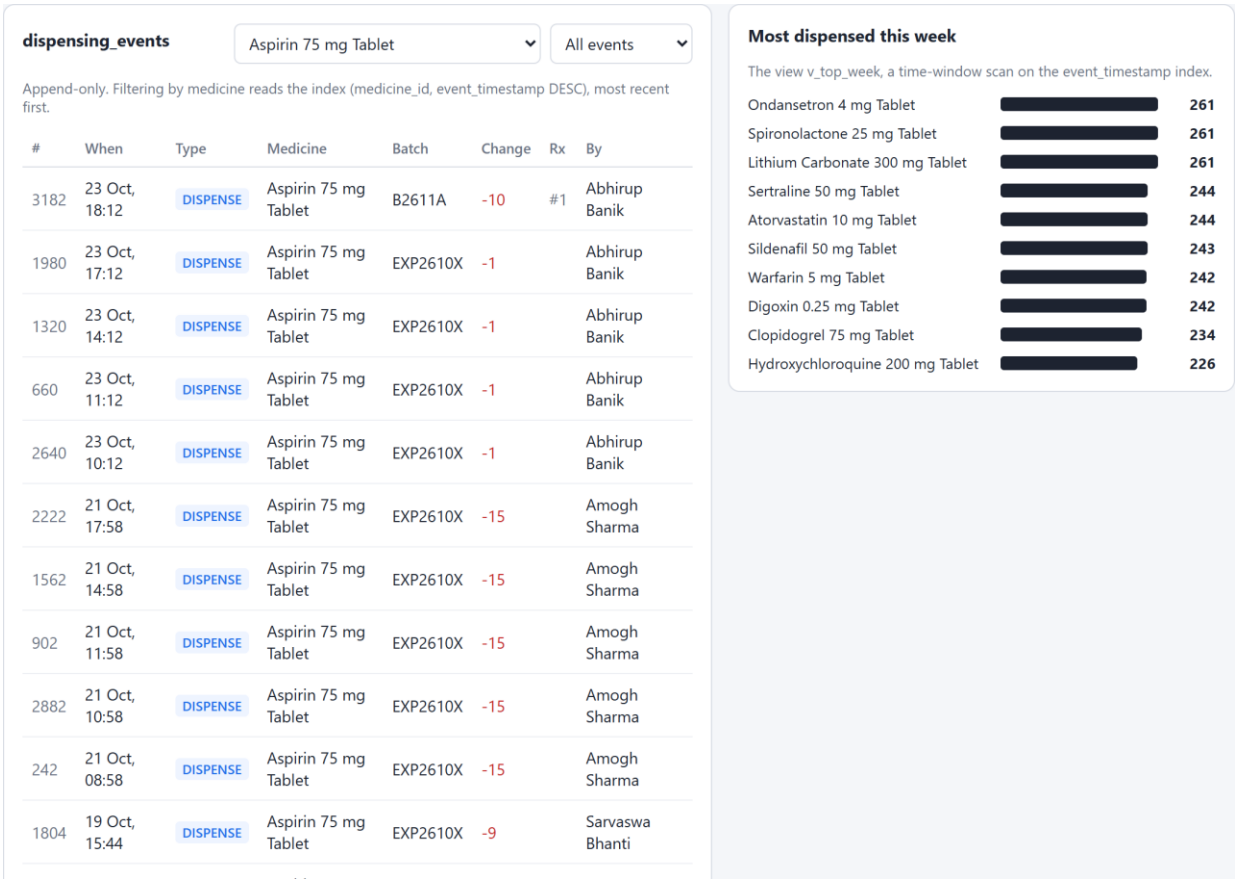


Figure 23: History of one medicine, newest first

5.4.5 Interaction Knowledge

The drug_interactions collection. Different documents carry different fields.

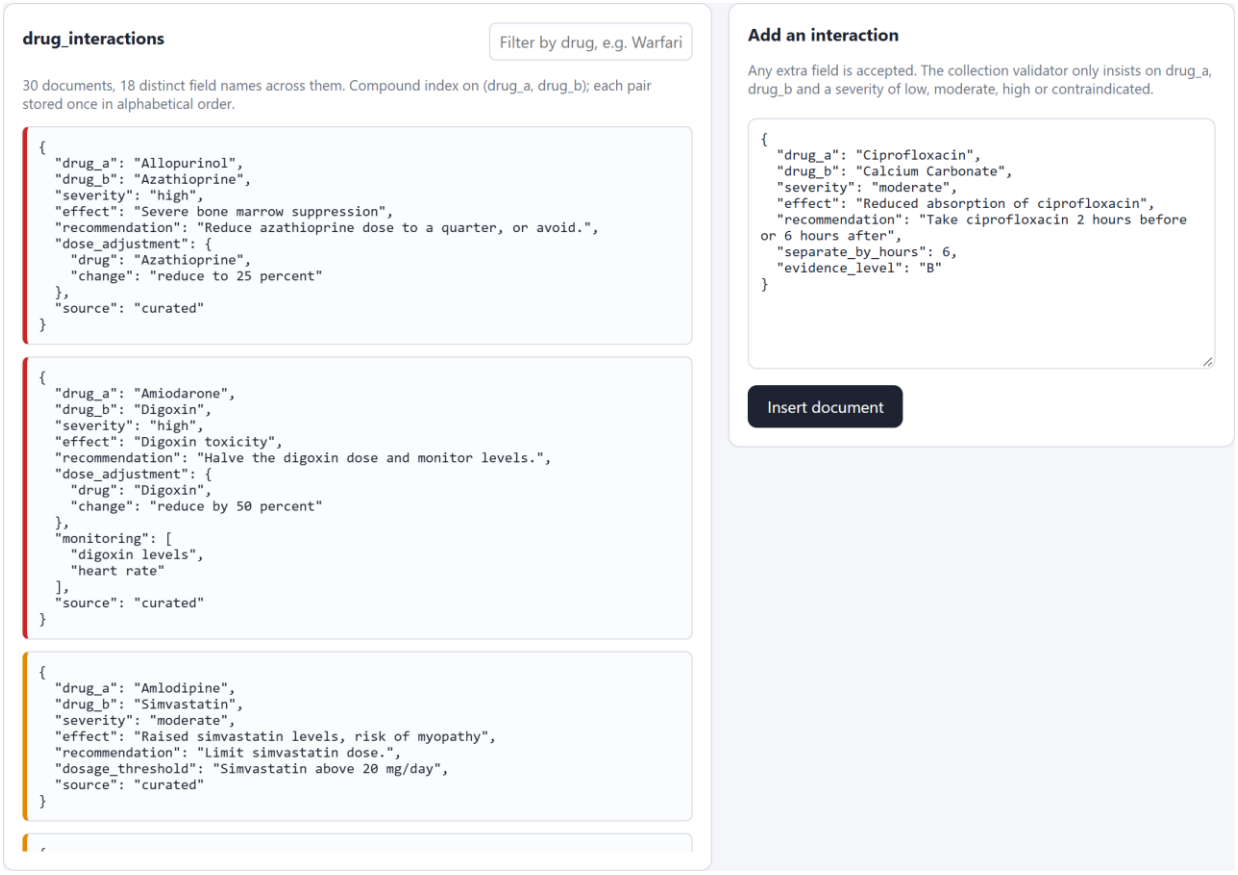


Figure 24: The drug_interactions collection; fields vary by pair

A new document with two fields that no other document has is accepted without any change to a schema.

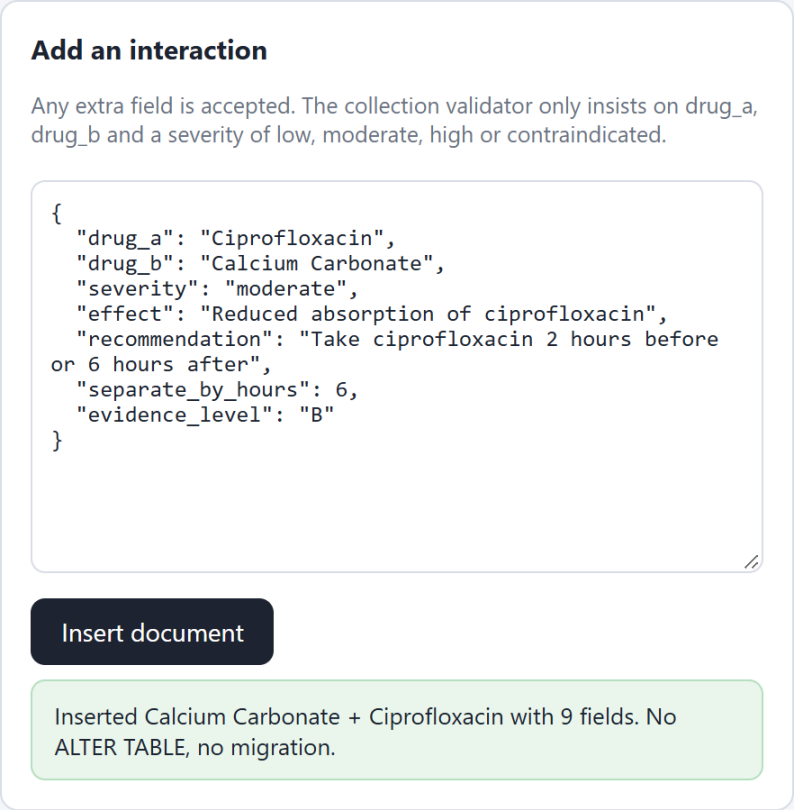


Figure 25: A document with new fields is accepted without any migration

A document without a severity is rejected by the collection validator.

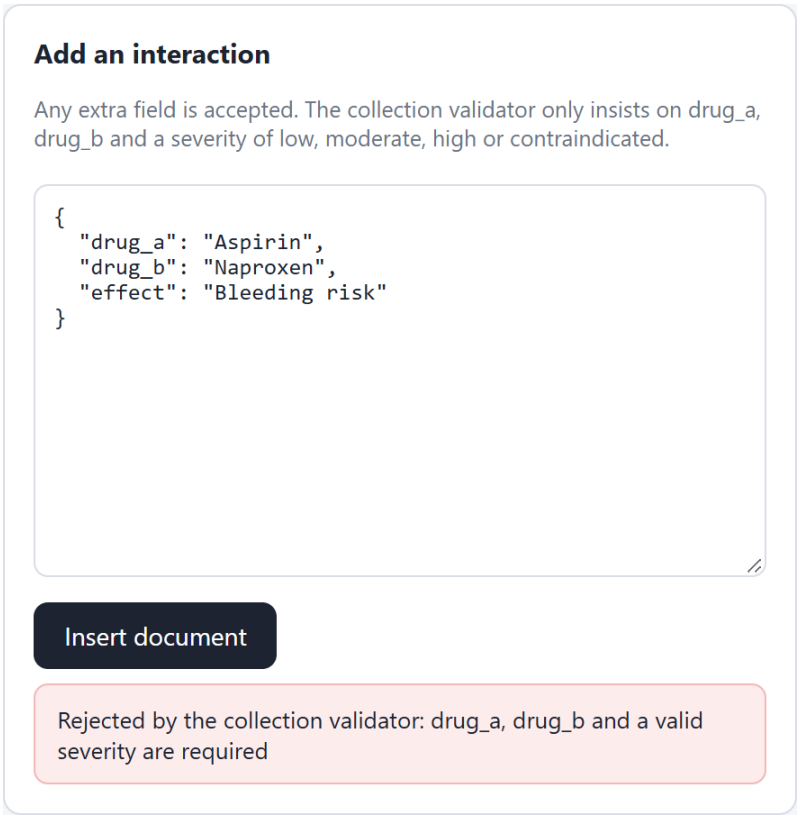


Figure 26: The $jsonSchema validator rejects a document without a severity

5.4.6 Under the Hood

Two pharmacists ask for the last unit. B waits on the row lock, gets it only after A commits, finds the batch empty and takes the next batch.

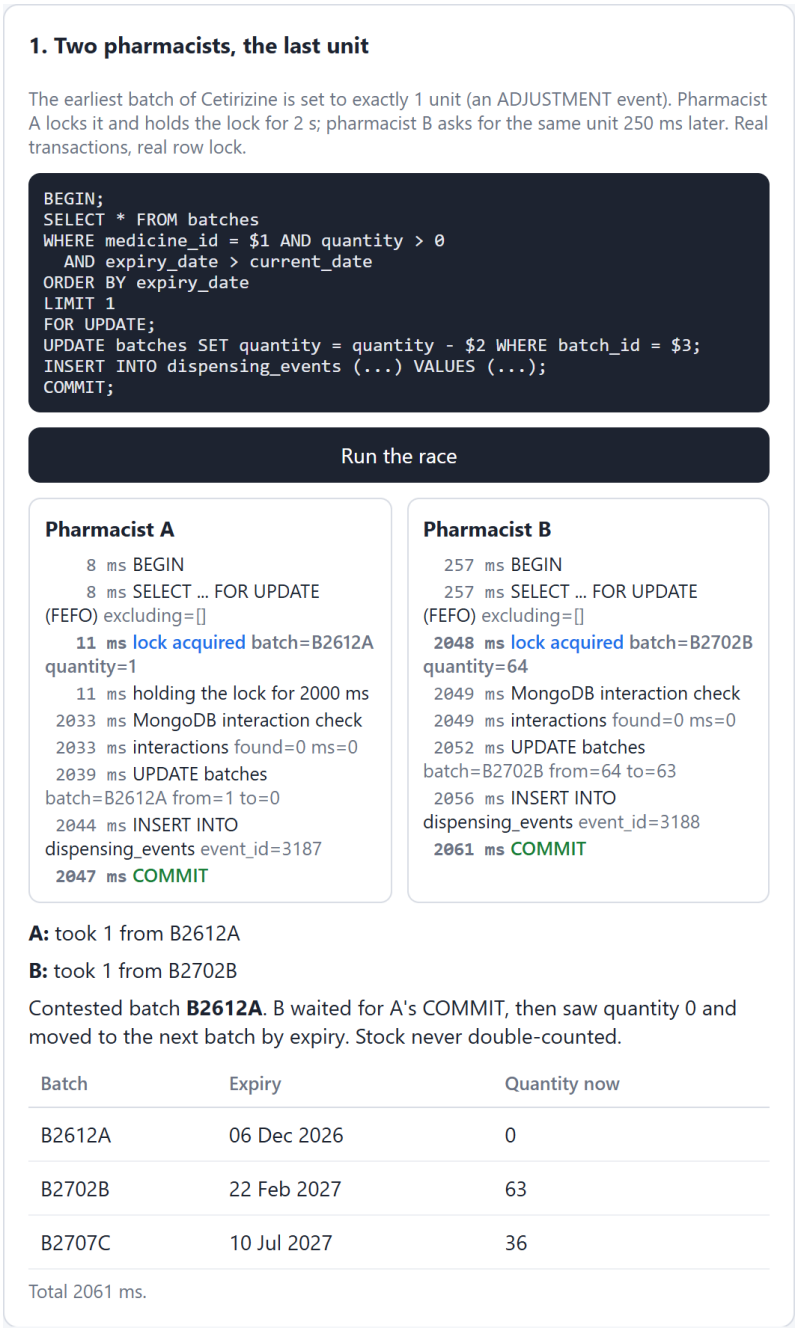


Figure 27: Two pharmacists, the last unit: the row lock serialises them

With MongoDB cut off, the dispense reaches the safety check inside the transaction, times out and rolls back.

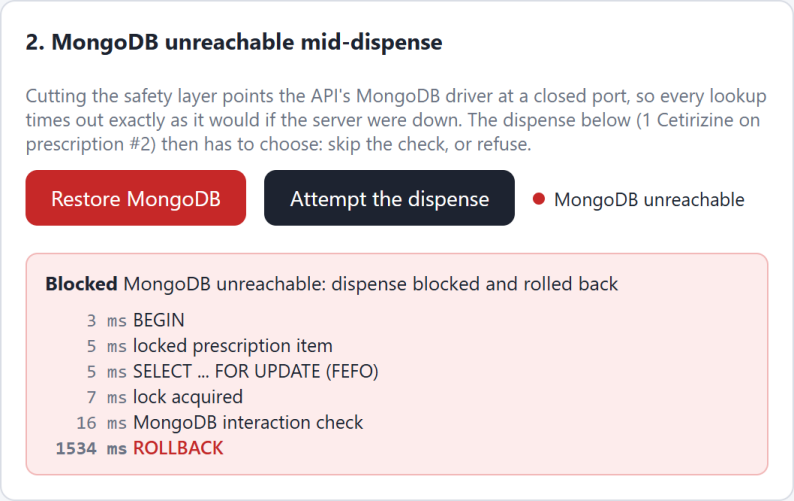


Figure 28: MongoDB unreachable: the dispense is blocked and rolled back

With MongoDB restored, the same dispense commits normally.

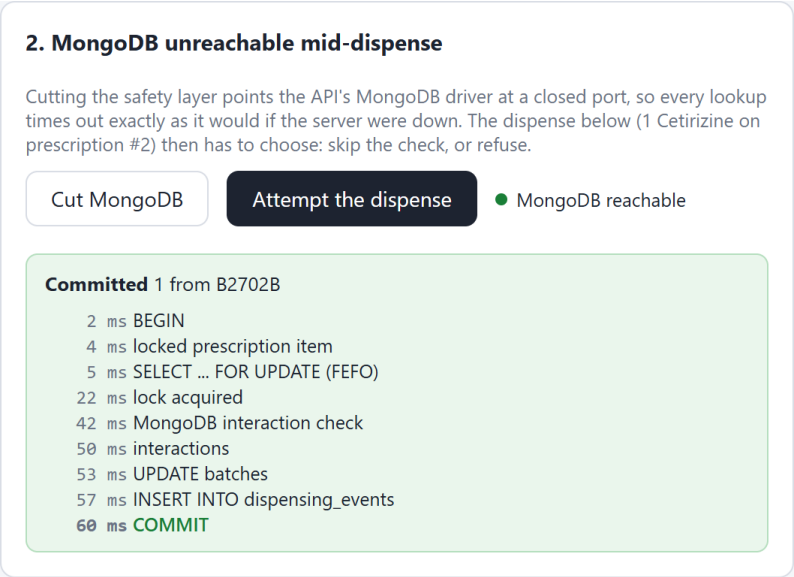


Figure 29: MongoDB restored: the same dispense commits

EXPLAIN ANALYZE of the FEFO query: an index scan on idx_batches_fefo against a sequential scan of the whole table.



Figure 30: EXPLAIN ANALYZE: index scan against sequential scan

The same UPDATE on the audit log is refused for both roles: by privilege for the API role and by the trigger for the owner.

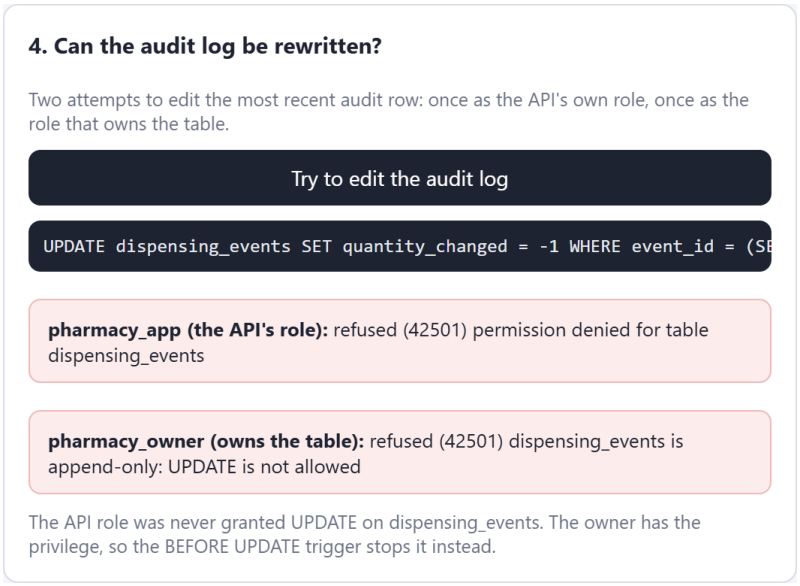


Figure 31: Both attempts to edit the audit log are refused

5.5 Source Code

The complete source code is available in the project repository at https://github.com/Awwbhirup/DBMS_LAB_PROJECT. The listings below contain the whole database layer and the code of the dispense transaction, which together hold every rule described in this report.

Listing 1: Tables, keys and constraints `(sql/01_schema.sql)`

```
CREATE TABLE users (
    user_id      SERIAL PRIMARY KEY,
    name         VARCHAR(80)  NOT NULL,
    role         VARCHAR(20)  NOT NULL CHECK (role IN ('pharmacist', 'admin'))
);

CREATE TABLE medicines (
    medicine_id  SERIAL PRIMARY KEY,
    name         VARCHAR(120) NOT NULL UNIQUE,
    generic_name VARCHAR(80)  NOT NULL,
    manufacturer VARCHAR(80)  NOT NULL,
    form         VARCHAR(30)  NOT NULL,
    strength     VARCHAR(30)  NOT NULL,
    pack         VARCHAR(40)  NOT NULL
);

CREATE TABLE batches (
    batch_id      SERIAL PRIMARY KEY,
    medicine_id   INT          NOT NULL REFERENCES medicines(medicine_id) ON DELETE
        RESTRICT,
    batch_number  VARCHAR(30)  NOT NULL,
    expiry_date   DATE         NOT NULL,
    quantity      INT          NOT NULL CHECK (quantity >= 0),
    received_date DATE         NOT NULL DEFAULT current_date,
    UNIQUE (medicine_id, batch_number),
    CHECK (expiry_date > received_date)
);

CREATE TABLE prescriptions (
    prescription_id SERIAL PRIMARY KEY,
    patient_name    VARCHAR(80) NOT NULL CHECK (length(trim(patient_name)) > 0),
    patient_age     INT         CHECK (patient_age BETWEEN 0 AND 120),
    doctor_name     VARCHAR(80) NOT NULL CHECK (length(trim(doctor_name)) > 0),
    issued_date     DATE        NOT NULL DEFAULT current_date
);

CREATE TABLE prescription_items (
    item_id             SERIAL PRIMARY KEY,
    prescription_id     INT NOT NULL REFERENCES prescriptions(prescription_id) ON DELETE
        RESTRICT,
    medicine_id         INT NOT NULL REFERENCES medicines(medicine_id) ON DELETE RESTRICT,
    quantity_prescribed INT NOT NULL CHECK (quantity_prescribed > 0),
    UNIQUE (prescription_id, medicine_id)
);

CREATE TABLE dispensing_events (
    event_id         BIGSERIAL   PRIMARY KEY,
    medicine_id      INT         NOT NULL REFERENCES medicines(medicine_id),
```

```
    batch_id         INT         NOT NULL REFERENCES batches(batch_id),
    prescription_id  INT         REFERENCES prescriptions(prescription_id),
    event_type       VARCHAR(30) NOT NULL CHECK (event_type IN ('DISPENSE', 'RECEIPT',
        'ADJUSTMENT')),
    quantity_changed INT         NOT NULL CHECK (quantity_changed <> 0),
    performed_by     INT         NOT NULL REFERENCES users(user_id),
    event_timestamp  TIMESTAMPTZ NOT NULL DEFAULT now(),
    CHECK (event_type <> 'DISPENSE' OR quantity_changed < 0),
    CHECK (event_type <> 'RECEIPT' OR quantity_changed > 0)
);
```

Listing 2: Indexes `(sql/02_indexes.sql)`

```
CREATE INDEX idx_batches_fefo
    ON batches (medicine_id, expiry_date)
    WHERE quantity > 0;


CREATE INDEX idx_events_medicine_time
    ON dispensing_events (medicine_id, event_timestamp DESC);


CREATE INDEX idx_events_time
    ON dispensing_events (event_timestamp);


CREATE INDEX idx_items_prescription
    ON prescription_items (prescription_id);


CREATE INDEX idx_medicines_name_lower
    ON medicines (lower(name) text_pattern_ops);


CREATE INDEX idx_medicines_generic_lower
    ON medicines (lower(generic_name) text_pattern_ops);
```

Listing 3: Append-only trigger on the audit log `(sql/03_audit_guard.sql)`

```
CREATE OR REPLACE FUNCTION reject_audit_change() RETURNS trigger AS $$
BEGIN
    RAISE EXCEPTION 'dispensing_events is append-only: % is not allowed', TG_OP
        USING ERRCODE = 'insufficient_privilege';
END;
$$ LANGUAGE plpgsql;


CREATE TRIGGER trg_events_no_update
    BEFORE UPDATE OR DELETE ON dispensing_events
    FOR EACH ROW EXECUTE FUNCTION reject_audit_change();


CREATE TRIGGER trg_events_no_truncate
    BEFORE TRUNCATE ON dispensing_events
    FOR EACH STATEMENT EXECUTE FUNCTION reject_audit_change();
```

Listing 4: Views `(sql/04_views.sql)`

```
CREATE VIEW v_stock AS
SELECT m.medicine_id,
       m.name,
       m.generic_name,
```

```
       COALESCE(SUM(b.quantity) FILTER (WHERE b.expiry_date > current_date), 0) AS
           usable_stock,
       COALESCE(SUM(b.quantity) FILTER (WHERE b.expiry_date <= current_date), 0) AS
           expired_stock,
       MIN(b.expiry_date) FILTER (WHERE b.quantity > 0 AND b.expiry_date > current_date) AS
           next_expiry,
       COUNT(b.batch_id) FILTER (WHERE b.quantity > 0) AS live_batches
FROM medicines m
LEFT JOIN batches b ON b.medicine_id = m.medicine_id
GROUP BY m.medicine_id, m.name, m.generic_name;


CREATE VIEW v_near_expiry AS
SELECT b.batch_id,
       b.batch_number,
       m.name,
       b.expiry_date,
       b.quantity,
       b.expiry_date - current_date AS days_left
FROM batches b
JOIN medicines m ON m.medicine_id = b.medicine_id
WHERE b.quantity > 0
  AND b.expiry_date <= current_date + 60
ORDER BY b.expiry_date;


CREATE VIEW v_top_week AS
SELECT m.medicine_id,
       m.name,
       -SUM(e.quantity_changed) AS units_dispensed,
       COUNT(*) AS dispenses
FROM dispensing_events e
JOIN medicines m ON m.medicine_id = e.medicine_id
WHERE e.event_type = 'DISPENSE'
  AND e.event_timestamp >= now() - interval '7 days'
GROUP BY m.medicine_id, m.name
ORDER BY units_dispensed DESC;


CREATE VIEW v_prescription_progress AS
SELECT pi.prescription_id,
       pi.medicine_id,
       pi.quantity_prescribed,
       COALESCE(-SUM(e.quantity_changed) FILTER (WHERE e.event_type = 'DISPENSE'), 0) AS
           quantity_dispensed
FROM prescription_items pi
LEFT JOIN dispensing_events e
       ON e.prescription_id = pi.prescription_id
      AND e.medicine_id = pi.medicine_id
GROUP BY pi.prescription_id, pi.medicine_id, pi.quantity_prescribed;
```

Listing 5: The preview and the confirm transaction `(server/dispense.js)`

```
import { pool, findInteractions, SafetyUnavailable } from "./db.js";


export class DispenseError extends Error {
  constructor(status, code, message, extra = {}) {
    super(message);
    this.status = status;
    this.code = code;
    this.extra = extra;
  }
```

```
}

const FEFO_SQL = `SELECT batch_id, batch_number, expiry_date, quantity
FROM batches
WHERE medicine_id = $1 AND quantity > 0 AND expiry_date > current_date
  AND batch_id <> ALL($2::int[])
ORDER BY expiry_date, batch_id
LIMIT 1
FOR UPDATE`;

export function fefoPlan(batches, quantity) {
  const plan = [];
  let left = quantity;
  for (const b of batches) {
    if (left <= 0) break;
    if (b.quantity <= 0 || b.expired) continue;
    const take = Math.min(b.quantity, left);
    plan.push({ batch_id: b.batch_id, batch_number: b.batch_number, expiry_date:
        b.expiry_date, take });
    left -= take;
  }
  return { plan, short_by: Math.max(left, 0) };
}

async function prescriptionContext(db, prescriptionId, medicineId) {
  const rx = await db.query(
    `SELECT pi.medicine_id, m.name, m.generic_name, p.quantity_prescribed,
       p.quantity_dispensed
     FROM prescription_items pi
     JOIN medicines m ON m.medicine_id = pi.medicine_id
     JOIN v_prescription_progress p ON p.prescription_id = pi.prescription_id AND
         p.medicine_id = pi.medicine_id
     WHERE pi.prescription_id = $1
     ORDER BY m.name`,
    [prescriptionId]
  );
  if (!rx.rowCount) throw new DispenseError(404, "no_prescription", "Prescription not
      found");
  const item = rx.rows.find((r) => r.medicine_id === medicineId);
  if (!item) throw new DispenseError(422, "not_on_prescription", "This medicine is not on
      the prescription");
  return { items: rx.rows, item };
}

function classify(found, generic) {
  const order = { contraindicated: 3, high: 2, moderate: 1, low: 0 };
  const list = found
    .map((d) => ({ ...d, _id: String(d._id), involves_this: d.drug_a === generic ||
        d.drug_b === generic }))
    .sort((a, b) => b.involves_this - a.involves_this || order[b.severity] -
        order[a.severity]);
  const blocking = list.filter((d) => d.involves_this && d.severity === "contraindicated");
  const needsAck = list.filter((d) => d.involves_this && d.severity !== "contraindicated"
      && d.severity !== "low");
  return { list, blocking, needsAck };
}

export async function preview({ prescription_id, medicine_id, quantity }) {
  const med = await pool.query("SELECT * FROM medicines WHERE medicine_id = $1",
```

```
      [medicine_id]);
  if (!med.rowCount) throw new DispenseError(404, "no_medicine", "Medicine not found");
  const medicine = med.rows[0];
  const batches = await pool.query(
    `SELECT batch_id, batch_number, expiry_date, quantity, received_date,
            expiry_date <= current_date AS expired,
            expiry_date - current_date AS days_left
     FROM batches
     WHERE medicine_id = $1 AND quantity > 0
     ORDER BY expiry_date, batch_id`,
    [medicine_id]
  );
  const { plan, short_by } = fefoPlan(batches.rows, quantity);
  let rx = null;
  let generics = [medicine.generic_name];
  if (prescription_id) {
    rx = await prescriptionContext(pool, prescription_id, medicine_id);
    generics = rx.items.map((i) => i.generic_name);
  }
  let safety;
  try {
    const r = await findInteractions(generics);
    const c = classify(r.found, medicine.generic_name);
    safety = { status: "ok", pairs_checked: r.pairs_checked, ms: r.ms, interactions:
        c.list, blocking: c.blocking.length > 0, needs_ack: c.needsAck.map((d) => d._id) };
  } catch (e) {
    if (!(e instanceof SafetyUnavailable)) throw e;
    safety = { status: "unavailable", message: e.message };
  }
  return {
    medicine,
    batches: batches.rows,
    plan,
    short_by,
    prescription: rx && { items: rx.items, remaining: rx.item.quantity_prescribed -
        rx.item.quantity_dispensed },
    safety
  };
}

export async function confirm(input, opts = {}) {
  const { prescription_id, medicine_id, quantity, user_id, acknowledged = [] } = input;
  const trace = opts.trace || (() => {});
  const holdMs = opts.holdMs || 0;
  if (!Number.isInteger(quantity) || quantity <= 0) throw new DispenseError(422,
      "bad_quantity", "Quantity must be a positive whole number");
  const client = await pool.connect();
  const taken = [];
  try {
    await client.query("BEGIN");
    trace("BEGIN");
    if (prescription_id) {
      await client.query(
        "SELECT item_id FROM prescription_items WHERE prescription_id = $1 AND medicine_id
            = $2 FOR UPDATE",
        [prescription_id, medicine_id]
      );
      trace("locked prescription item");
    }
    let left = quantity;
    const seen = [];
```

```
    while (left > 0) {
      trace("SELECT ... FOR UPDATE (FEFO)", { excluding: [...seen] });
      const r = await client.query(FEFO_SQL, [medicine_id, seen]);
      if (!r.rowCount) {
        throw new DispenseError(409, "insufficient_stock", `Not enough unexpired stock:
            short by ${left}`, { short_by: left });
      }
      const b = r.rows[0];
      trace("lock acquired", { batch: b.batch_number, quantity: b.quantity });
      const take = Math.min(b.quantity, left);
      taken.push({ batch_id: b.batch_id, batch_number: b.batch_number, expiry_date:
          b.expiry_date, before: b.quantity, take });
      seen.push(b.batch_id);
      left -= take;
    }
    if (holdMs) {
      trace(`holding the lock for ${holdMs} ms`);
      await new Promise((res) => setTimeout(res, holdMs));
    }
    const med = await client.query("SELECT generic_name FROM medicines WHERE medicine_id =
        $1", [medicine_id]);
    const generic = med.rows[0].generic_name;
    let generics = [generic];
    if (prescription_id) {
      const rx = await prescriptionContext(client, prescription_id, medicine_id);
      const remaining = rx.item.quantity_prescribed - rx.item.quantity_dispensed;
      if (quantity > remaining) {
        throw new DispenseError(409, "over_prescription", `Only ${remaining} left to
            dispense on this prescription`, { remaining });
      }
      generics = rx.items.map((i) => i.generic_name);
    }
    trace("MongoDB interaction check");
    let check;
    try {
      check = await findInteractions(generics);
    } catch (e) {
      if (e instanceof SafetyUnavailable) {
        throw new DispenseError(503, "safety_unavailable", "MongoDB unreachable: dispense
            blocked and rolled back");
      }
      throw e;
    }
    const c = classify(check.found, generic);
    trace("interactions", { found: check.found.length, ms: check.ms });
    if (c.blocking.length) {
      throw new DispenseError(409, "contraindicated", "Contraindicated combination:
          dispense refused", { interactions: c.blocking });
    }
    const missing = c.needsAck.filter((d) => !acknowledged.includes(d._id));
    if (missing.length) {
      throw new DispenseError(409, "needs_acknowledgement", "Interaction warning must be
          acknowledged", { interactions: missing });
    }
    const events = [];
    for (const t of taken) {
      await client.query("UPDATE batches SET quantity = quantity - $1 WHERE batch_id = $2",
          [t.take, t.batch_id]);
      trace("UPDATE batches", { batch: t.batch_number, from: t.before, to: t.before -
          t.take });
      const ev = await client.query(
        `INSERT INTO dispensing_events (medicine_id, batch_id, prescription_id, event_type,
```

```
            quantity_changed, performed_by)
         VALUES ($1, $2, $3, 'DISPENSE', $4, $5)
         RETURNING event_id, event_timestamp`,
        [medicine_id, t.batch_id, prescription_id || null, -t.take, user_id]
      );
      events.push(ev.rows[0]);
      trace("INSERT INTO dispensing_events", { event_id: ev.rows[0].event_id });
    }
    await client.query("COMMIT");
    trace("COMMIT");
    return { ok: true, taken, events, interactions: c.list };
  } catch (e) {
    await client.query("ROLLBACK").catch(() => {});
    trace("ROLLBACK", { reason: e.code || e.message });
    throw e;
  } finally {
    client.release();
  }
}
```

Listing 6: Connection pools, MongoDB client and the interaction lookup `(server/db.js)`

```
import "dotenv/config";
import pg from "pg";
import { MongoClient } from "mongodb";

const env = process.env;

pg.types.setTypeParser(1082, (v) => v);
pg.types.setTypeParser(20, (v) => Number(v));

export const pool = new pg.Pool({
  host: env.PGHOST,
  port: Number(env.PGPORT),
  database: env.PGDATABASE,
  user: env.PGUSER,
  password: env.PGPASSWORD,
  max: 12
});

export const ownerPool = new pg.Pool({
  host: env.PGHOST,
  port: Number(env.PGPORT),
  database: env.PGDATABASE,
  user: env.PGOWNER_USER,
  password: env.PGOWNER_PASSWORD,
  max: 2
});

for (const p of [pool, ownerPool]) p.on("error", () => {});

const DEAD_URL = "mongodb://127.0.0.1:1";
let mongo = null;
let outage = false;

export function mongoOutage() {
  return outage;
```

```
}

export async function setMongoOutage(on) {
  outage = on;
  if (mongo) {
    await mongo.close().catch(() => {});
    mongo = null;
  }
}

export async function interactionsCollection() {
  if (!mongo) {
    mongo = new MongoClient(outage ? DEAD_URL : env.MONGO_URL, {
      serverSelectionTimeoutMS: 1500,
      connectTimeoutMS: 1500
    });
    try {
      await mongo.connect();
    } catch (e) {
      mongo = null;
      throw e;
    }
  }
  return mongo.db(env.MONGO_DB).collection("drug_interactions");
}

export class SafetyUnavailable extends Error {
  constructor(cause) {
    super("Interaction check could not be completed: MongoDB unreachable");
    this.cause = cause;
  }
}

export async function findInteractions(genericNames) {
  const names = [...new Set(genericNames)].sort((a, b) => a.localeCompare(b));
  const pairs = [];
  for (let i = 0; i < names.length; i++) {
    for (let j = i + 1; j < names.length; j++) pairs.push({ drug_a: names[i], drug_b:
        names[j] });
  }
  if (!pairs.length) return { pairs_checked: 0, found: [], ms: 0 };
  const t = performance.now();
  let col;
  try {
    col = await interactionsCollection();
    const found = await col.find({ $or: pairs }, { projection: { added_at: 0 } }).toArray();
    return { pairs_checked: pairs.length, found, ms: Math.round((performance.now() - t) *
        10) / 10 };
  } catch (e) {
    await setMongoOutage(outage);
    throw new SafetyUnavailable(e);
  }
}
```

Listing 7: Setup: roles, privileges, MongoDB validator and index `(scripts/setup.js)`

```
import "dotenv/config";
import fs from "node:fs";
import path from "node:path";
```

```
import { fileURLToPath } from "node:url";
import pg from "pg";
import { MongoClient } from "mongodb";
import interactions from "../mongo/interactions.js";

const root = path.resolve(path.dirname(fileURLToPath(import.meta.url)), "..");
const env = process.env;
const ident = (s) => '"' + s.replace(/"/g, '""') + '"';
const lit = (s) => "'" + s.replace(/'/g, "''") + "'";

function log(msg) {
  console.log(new Date().toISOString().slice(11, 19) + "  " + msg);
}

async function connect(database, user, password) {
  const c = new pg.Client({ host: env.PGHOST, port: Number(env.PGPORT), database, user,
      password });
  await c.connect();
  return c;
}

async function setupPostgres() {
  const su = await connect("postgres", env.PG_SUPERUSER, env.PG_SUPERPASS);
  for (const [role, pw] of [[env.PGOWNER_USER, env.PGOWNER_PASSWORD], [env.PGUSER,
      env.PGPASSWORD]]) {
    const exists = await su.query("SELECT 1 FROM pg_roles WHERE rolname = $1", [role]);
    const verb = exists.rowCount ? "ALTER" : "CREATE";
    await su.query(`${verb} ROLE ${ident(role)} LOGIN PASSWORD ${lit(pw)}`);
  }
  await su.query(`DROP DATABASE IF EXISTS ${ident(env.PGDATABASE)} WITH (FORCE)`);
  await su.query(`CREATE DATABASE ${ident(env.PGDATABASE)} OWNER
      ${ident(env.PGOWNER_USER)}`);
  await su.end();
  log("postgres: database " + env.PGDATABASE + " created");

  const owner = await connect(env.PGDATABASE, env.PGOWNER_USER, env.PGOWNER_PASSWORD);
  await owner.query("REVOKE ALL ON SCHEMA public FROM PUBLIC");
  const files = fs.readdirSync(path.join(root, "sql")).filter((f) =>
      f.endsWith(".sql")).sort();
  for (const f of files) {
    const t = Date.now();
    await owner.query(fs.readFileSync(path.join(root, "sql", f), "utf8"));
    log(`postgres: ${f} (${Date.now() - t} ms)`);
  }
  const app = ident(env.PGUSER);
  await owner.query(`GRANT USAGE ON SCHEMA public TO ${app}`);
  await owner.query(`GRANT SELECT ON ALL TABLES IN SCHEMA public TO ${app}`);
  await owner.query(`GRANT INSERT, UPDATE ON batches, prescriptions, prescription_items TO
      ${app}`);
  await owner.query(`GRANT INSERT ON dispensing_events TO ${app}`);
  await owner.query(`GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO ${app}`);
  const counts = await owner.query(`
    SELECT (SELECT count(*) FROM medicines) AS medicines,
           (SELECT count(*) FROM batches WHERE quantity > 0) AS live_batches,
           (SELECT count(*) FROM batches) AS all_batches,
           (SELECT count(*) FROM prescriptions) AS prescriptions,
           (SELECT count(*) FROM dispensing_events) AS events`);
  log("postgres: " + JSON.stringify(counts.rows[0]));
  await owner.end();
```

```
}


function normalise(doc) {
  const [a, b] = [doc.drug_a, doc.drug_b].sort((x, y) => x.localeCompare(y));
  return { ...doc, drug_a: a, drug_b: b };
}


async function setupMongo() {
  const client = new MongoClient(env.MONGO_URL, { serverSelectionTimeoutMS: 4000 });
  await client.connect();
  const db = client.db(env.MONGO_DB);
  await db.dropDatabase();
  await db.createCollection("drug_interactions", {
    validator: {
      $jsonSchema: {
        bsonType: "object",
        required: ["drug_a", "drug_b", "severity"],
        properties: {
          drug_a: { bsonType: "string" },
          drug_b: { bsonType: "string" },
          severity: { enum: ["low", "moderate", "high", "contraindicated"] }
        }
      }
    }
  });
  const col = db.collection("drug_interactions");
  await col.createIndex({ drug_a: 1, drug_b: 1 }, { unique: true, name: "pair" });
  const docs = interactions.map(normalise).map((d) => ({ ...d, source: d.source ??
      "curated", added_at: new Date() }));
  await col.insertMany(docs);
  log(`mongo: ${docs.length} interaction documents, index on (drug_a, drug_b)`);
  await client.close();
}


const only = process.argv[2];
try {
  if (only !== "mongo") await setupPostgres();
  if (only !== "postgres") await setupMongo();
  log("done");
} catch (e) {
  console.error(e.message);
  process.exit(1);
}
```

6. CONCLUSION

This project shows that the two halves of a pharmacy's data are best kept in two different databases, and that they can work together without giving up correctness. PostgreSQL enforces every rule that has to be exact. Keys and CHECK constraints make invalid data impossible to store. A single short transaction with row locks makes each dispense atomic and safe when several pharmacists work at once. A partial index keeps the FEFO choice fast however much history builds up. A trigger, together with the privileges of the API role, keeps the audit log append-only.

MongoDB holds the interaction knowledge, whose shape changes from pair to pair, with a validator that still sets a minimum standard for every document. The dispense refuses to go ahead whenever that knowledge cannot be checked, so the flexibility of the document store never weakens the safety of the system.

Each of these properties was demonstrated on the running system and measured, and the 24 automated checks confirm them on every run. In future the system could add login with roles in the interface, reorder alerts when stock runs low, and interaction data loaded from a published drug database in place of the curated seed.

7. REFERENCES

[1] P. J. Sadalage and M. Fowler, NoSQL Distilled: A Brief Guide to the Emerging World of Polyglot Persistence. Upper Saddle River, NJ, USA: Addison-Wesley, 2012.

[2] M. Kleppmann, Designing Data-Intensive Applications. Sebastopol, CA, USA: O'Reilly Media, 2017.

[3] E. F. Codd, "A relational model of data for large shared data banks," Communications of the ACM, vol. 13, no. 6, pp. 377-387, 1970.

[4] J. Gray and A. Reuter, Transaction Processing: Concepts and Techniques. San Mateo, CA, USA: Morgan Kaufmann, 1992.

[5] H. Berenson, P. Bernstein, J. Gray, J. Melton, E. O'Neil and P. O'Neil, "A critique of ANSI SQL isolation levels," in Proc. ACM SIGMOD Int. Conf. on Management of Data, 1995, pp. 1-10.

[6] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation, Chapter 13: Concurrency Control." Available: https://www.postgresql.org/docs/17/mvcc.html

[7] A. Silberschatz, H. F. Korth and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019.

[8] The PostgreSQL Global Development Group, "PostgreSQL 17 Documentation, Section 11.8: Partial Indexes." Available: https://www.postgresql.org/docs/17/indexes-partial.html

[9] M. Stonebraker and U. Cetintemel, "'One size fits all': an idea whose time has come and gone," in Proc. 21st Int. Conf. on Data Engineering (ICDE), 2005, pp. 2-11.

[10] R. Hecht and S. Jablonski, "NoSQL evaluation: a use case oriented survey," in Proc. Int. Conf. on Cloud and Service Computing, 2011, pp. 336-341.

[11] MongoDB, Inc., "MongoDB Manual: Schema Validation." Available: https://www.mongodb.com/docs/manual/core/schema-validation/

[12] D. S. Wishart et al., "DrugBank 5.0: a major update to the DrugBank database for 2018," Nucleic Acids Research, vol. 46, no. D1, pp. D1074-D1082, 2018.

[13] World Health Organization, "Good storage and distribution practices for medical products," WHO Technical Report Series, no. 1025, Annex 7, 2020.